Labsheet-1

Real-World Data Collection, Cleaning and Preprocessing using Python

In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler

# 1. Load dataset
df = pd.read_csv("WA_Fn-UseC_-HR-Employee-Attrition-selected-columns.csv")

# 2. Inspect
print(df.head())
print("\nShape:", df.shape)
print("\nMissing Values:\n", df.isnull().sum())

# 3. Handle missing values
for col in df.select_dtypes(include=np.number):
    df[col] = df[col].fillna(df[col].median())

for col in df.select_dtypes(include="object"):
    df[col] = df[col].fillna(df[col].mode()[0])

# 4. Remove duplicates
df.drop_duplicates(inplace=True)

# 5. Clean categorical values
for col in df.select_dtypes(include="object"):
    df[col] = df[col].str.strip()

# 6. Feature Engineering
if "Age" in df.columns:
    df["AgeGroup"] = pd.cut(
        df["Age"],
        bins=[0, 25, 35, 45, 60, 100],
        labels=["Young", "Early", "Middle", "Senior", "Very Senior"]
    )

# 7. Outlier detection and treatment
for col in ["Age", "MonthlyIncome", "TotalWorkingYears"]:
    if col in df.columns:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        df[col] = df[col].clip(
            Q1 - 1.5 * IQR,
            Q3 + 1.5 * IQR
        )

# 8. Label Encoding
le = LabelEncoder()

for col in ["Attrition", "Gender", "OverTime"]:
    if col in df.columns:
        df[col] = le.fit_transform(df[col].astype(str))
```

```
# One-Hot Encoding
df = pd.get_dummies(df, drop_first=True)

# 9. Standardization
cols = [
    col for col in ["Age", "MonthlyIncome", "TotalWorkingYears"]
    if col in df.columns
]

if cols:
    df[cols] = StandardScaler().fit_transform(df[cols])

# 10. Save dataset
df.to_csv("IBM_HR_Preprocessed.csv", index=False)

# Final output
print("\nPreprocessing Completed!")
print("Final Shape:", df.shape)
print("Missing Values:", df.isnull().sum().sum())
print("\nFinal Data:")
print(df.head())
```

```
   Age Attrition     BusinessTravel  DailyRate              Department  \
0   41       Yes      Travel_Rarely       1102                   Sales
1   49        No  Travel_Frequently        279  Research & Development
2   37       Yes      Travel_Rarely       1373  Research & Development
3   33        No  Travel_Frequently       1392  Research & Development
4   27        No      Travel_Rarely        591  Research & Development

   DistanceFromHome  Education EducationField  EmployeeCount  EmployeeNumber
0                 1          2  Life Sciences              1               1
1                 8          1  Life Sciences              1               2
2                 2          2          Other              1               4
3                 3          4  Life Sciences              1               5
4                 2          1        Medical              1               7

Shape: (1470, 10)

Missing Values:
 Age                 0
Attrition            0
BusinessTravel       0
DailyRate            0
Department           0
DistanceFromHome     0
Education            0
EducationField       0
EmployeeCount        0
EmployeeNumber       0
dtype: int64

Preprocessing Completed!
Final Shape: (1470, 20)
Missing Values: 0

Final Data:
        Age  Attrition  DailyRate  DistanceFromHome  Education  EmployeeCount  \
0  0.446350          1       1102                 1          2              1
1  1.322365          0        279                 8          1              1
2  0.008343          1       1373                 2          2              1
3 -0.429664          0       1392                 3          4              1
4 -1.086676          0        591                 2          1              1

   EmployeeNumber  BusinessTravel_Travel_Frequently  \
0               1                             False
1               2                              True
2               4                             False
3               5                              True
4               7                             False

   BusinessTravel_Travel_Rarely  Department_Research & Development  \
0                          True                              False
1                         False                               True
2                          True                               True
3                         False                               True
4                          True                               True

   Department_Sales  EducationField_Life Sciences  EducationField_Marketing  \
0              True                          True                     False
1             False                          True                     False
2             False                         False                     False
3             False                          True                     False
```

```
4             False                          False                       False

   EducationField_Medical  EducationField_Other  \
0                   False                 False
1                   False                 False
2                   False                  True
3                   False                 False
4                    True                 False

   EducationField_Technical Degree  AgeGroup_Early  AgeGroup_Middle  \
0                           False           False             True
1                           False           False            False
2                           False           False             True
3                           False            True            False
4                           False            True            False

   AgeGroup_Senior  AgeGroup_Very Senior
0            False                 False
1             True                 False
2            False                 False
3            False                 False
4            False                 False
```

Experiment 1: Real-World Data Collection, Cleaning and Preprocessing using Python

Questions & Answers

Q1. Why is data preprocessing considered one of the most important phases in data analytics?

- **Ensures Data Quality:** Real-world datasets contain noise, missing values, duplicates, and outliers that ruin analytical accuracy.
- **Improves Model Accuracy:** High-quality preprocessed data prevents biased learning and enables machine learning algorithms to converge faster with better accuracy.

Q2. Explain different methods of handling missing values with suitable examples.

- **Deletion:** Dropping rows or columns with excessive missing data (e.g., dropping a column missing 80% of its records).
- **Imputation (Mean/Median):** Replacing missing numerical values with the mean (for normal distributions) or median (for skewed data, like replacing missing Age with 28).
- **Categorical Imputation (Mode):** Replacing missing categorical values with the most frequent value (e.g., filling missing Embarked with 'S').

Q3. Differentiate between Label Encoding and One-Hot Encoding.

- **Label Encoding:** Assigns an integer value to each category (e.g., Low=0, Medium=1, High=2). Best for ordinal variables where rank matters.
- **One-Hot Encoding:** Creates separate binary columns (0 or 1) for each category. Best for nominal variables with no inherent order (e.g., Red, Blue, Green) to prevent false numerical ordering assumptions.

Q4. What are outliers? How can they affect analytical results?

- **Definition:** Outliers are data points that differ significantly from other observations in the dataset.
- **Effect:** They skew mean and variance calculations, distort regression lines, and negatively impact distance-based machine learning algorithms (like k-NN or K-Means).

Q5. Explain the difference between normalization and standardization.

- **Normalization (Min-Max Scaling):** Scales features to a fixed range, typically [0, 1].
- **Standardization (Z-Score Normalization):** Centers data around a mean of 0 with a standard deviation of 1.

Q6. Why should duplicate records be removed before analysis?

- **Prevents Data Bias:** Prevents giving double weight to identical records, which skews statistical metrics.
- **Prevents Overfitting:** Avoids data leakage between training and testing split sets during model evaluation.

Q7. What is feature engineering? Give two practical examples.

- **Definition:** The process of using domain knowledge to create new input variables/features from raw data to enhance model performance.
- **Example 1:** Combining `SibSp` (siblings/spouses) and `Parch` (parents/children) to create `FamilySize`.
- **Example 2:** Extracting `IsAlone` (binary indicator) or converting raw timestamps into `HourOfDay` or `IsWeekend`.

Q8. Which preprocessing techniques would you apply to the IBM HR Employee Attrition dataset and why?

- **One-Hot Encoding:** Convert categorical attributes like `Department` and `JobRole` into numerical binary columns.
- **Outlier Capping:** Apply Interquartile Range (IQR) capping on skewed financial variables like `MonthlyIncome`.
- **Standardization:** Scale continuous numerical attributes like `Age`, `TotalWorkingYears`, and `MonthlyIncome`.

Q9. How does poor-quality data affect machine learning model performance?

- Leads to "Garbage In, Garbage Out" (GIGO) where model predictions become unreliable.
- Introduces algorithmic bias, high variance, and poor generalization performance on unseen real-world test data.

Q10. Name any three Python libraries commonly used for data preprocessing.

• **Pandas:** For data manipulation, missing value handling, and duplicate removal.

• **NumPy:** For vectorized numerical operations and mathematical array transformations.

• **Scikit-Learn (sklearn.preprocessing):** For feature scaling, normalization, and categorical encoding.

```
Please choose your dataset file:
```
Choose Files Sample - Superstore.csv

Sample - Superstore.csv(text/csv) - 2287806 bytes, last modified: 9/12/2026 - 100% done

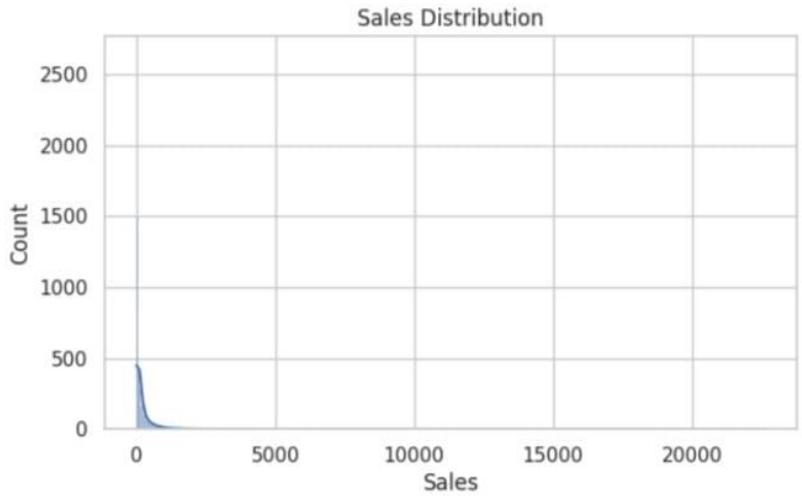
```
Saving Sample - Superstore.csv to Sample - Superstore (1).csv

Dataset Loaded Successfully!
Shape: (9994, 21)

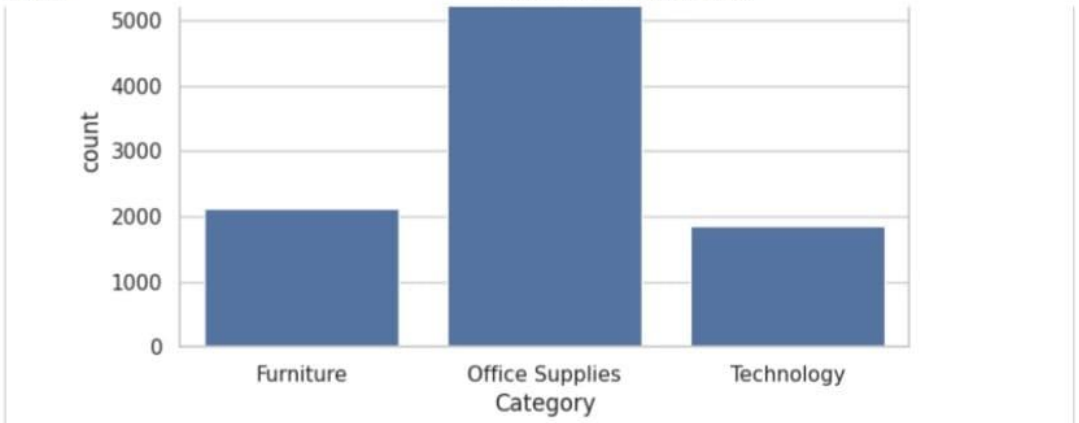
Columns:
['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID', 'Customer Name', 'Se

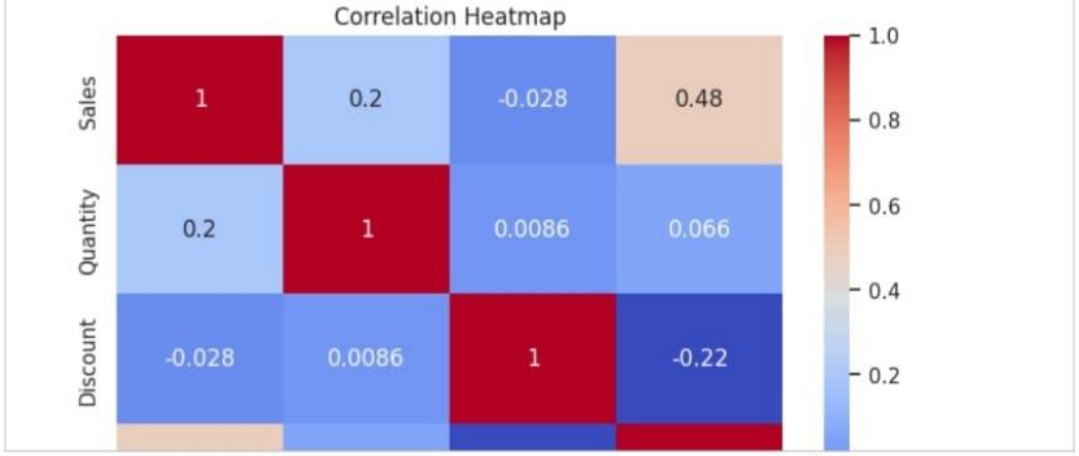
Data Types:
Row ID             int64
Order ID          object
Order Date        object
Ship Date         object
Ship Mode         object
Customer ID       object
Customer Name     object
Segment           object
Country           object
City              object
State             object
Postal Code        int64
Region            object
Product ID        object
Category          object
Sub-Category      object
Product Name      object
Sales            float64
Quantity           int64
Discount         float64
Profit           float64
dtype: object

Summary:
              Sales     Quantity     Discount       Profit
count   9994.000000  9994.000000  9994.000000  9994.000000
mean     229.858001     3.789574     0.156203    28.656896
std      623.245101     2.225110     0.206452   234.260108
min        0.444000     1.000000     0.000000 -6599.978000
25%       17.280000     2.000000     0.000000     1.728750
50%       54.490000     3.000000     0.200000     8.666500
75%      209.940000     5.000000     0.200000    29.364000
max    22638.480000    14.000000     0.800000  8399.976000
```






5000
4000
3000
2000
1000
0
count
Furniture
Office Supplies
Technology
Category


Correlation Heatmap
Sales
Quantity
Discount
1
0.2
-0.028
0.48
0.2
1
0.0086
0.066
-0.028
0.0086
1
-0.22
1.0
0.8
0.6
0.4
0.2

In [2]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Step 1: Load dataset
df = pd.read_csv("Sample - Superstore.csv", encoding="latin1")

print("Dataset loaded successfully!")

print("\nFirst 5 Records:")
print(df.head())


# Step 2: Check dataset
print("\nDataset Shape:")
print(df.shape)

print("\nDataset Information:")
df.info()

print("\nMissing Values:")
print(df.isnull().sum())


# Step 3: Bar Chart - Sales by Category
plt.figure(figsize=(8, 5))

sns.barplot(
    data=df,
    x="Category",
    y="Sales",
    estimator="sum"
)

plt.title("Total Sales by Category")
plt.xlabel("Category")
plt.ylabel("Total Sales")

plt.tight_layout()
plt.show()


# Step 4: Convert Order Date
# format="mixed" handles different date formats in the dataset

df["Order Date"] = pd.to_datetime(
    df["Order Date"],
    format="mixed"
)

# Sales by Year
year_data = df.groupby(
    df["Order Date"].dt.year
)["Sales"].sum()


# Step 5: Line Chart - Sales by Year
```

```
plt.figure(figsize=(10, 5))

plt.plot(
    year_data.index,
    year_data.values,
    marker="o"
)

plt.title("Total Sales by Year")
plt.xlabel("Year")
plt.ylabel("Total Sales")

plt.grid(True)

plt.tight_layout()
plt.show()


# Step 6: Histogram - Sales Distribution
plt.figure(figsize=(8, 5))

plt.hist(
    df["Sales"],
    bins=20
)

plt.title("Distribution of Sales")
plt.xlabel("Sales")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()


# Step 7: Box Plot - Profit by Category
plt.figure(figsize=(8, 5))

sns.boxplot(
    data=df,
    x="Category",
    y="Profit"
)

plt.title("Profit Distribution by Category")
plt.xlabel("Category")
plt.ylabel("Profit")

plt.tight_layout()
plt.show()


# Step 8: Scatter Plot - Sales vs Profit
plt.figure(figsize=(9, 5))

sns.scatterplot(
    data=df,
    x="Sales",
    y="Profit",
    hue="Category"
)
```

```
plt.title("Sales vs Profit")
plt.xlabel("Sales")
plt.ylabel("Profit")

plt.tight_layout()
plt.show()


# Step 9: Correlation Heatmap
numeric_data = df.select_dtypes(
    include="number"
)

correlation = numeric_data.corr()

plt.figure(figsize=(10, 7))

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap")

plt.tight_layout()
plt.show()


# Step 10: Customized Visualization - Sales by Region
region_sales = df.groupby(
    "Region"
)["Sales"].sum().sort_values(ascending=False)

plt.figure(figsize=(8, 5))

bars = plt.bar(
    region_sales.index,
    region_sales.values
)

plt.title("Sales Distribution by Region")
plt.xlabel("Region")
plt.ylabel("Total Sales")


# Display values on bars
for bar in bars:

    height = bar.get_height()

    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height,
        f"{height:.0f}",
        ha="center",
        va="bottom"
    )
```

```
plt.tight_layout()
plt.show()


# Step 11: Visualization Summary
print("\nVisualization Summary")
print("-" * 40)

print("1. The bar chart shows total sales for each product category.")

print("2. The line chart shows sales trends across different years.")

print("3. The histogram shows the distribution of sales values.")

print("4. The box plot compares profit distribution across categories.")

print("5. The scatter plot shows the relationship between sales and profit.")

print("6. The heatmap shows correlations between numerical variables.")

print("7. The customized chart compares sales across different regions.")


# Step 12: Business Insights
print("\nBusiness Insights")
print("-" * 40)

print(
    "Total Sales:",
    round(df["Sales"].sum(), 2)
)

print(
    "Total Profit:",
    round(df["Profit"].sum(), 2)
)

print(
    "Total Orders:",
    df["Order ID"].nunique()
)

print(
    "Total Customers:",
    df["Customer ID"].nunique()
)

print(
    "Best Category:",
    df.groupby("Category")["Sales"].sum().idxmax()
)

print(
    "Best Region:",
    df.groupby("Region")["Sales"].sum().idxmax()
)
```

```
# Step 13: Completion Message
print("\nExperiment completed successfully!")
```

```
Dataset loaded successfully!

First 5 Records:
   Row ID        Order ID  Order Date   Ship Date       Ship Mode Customer ID  \
0       1  CA-2016-152156  11-08-2016  11-11-2016    Second Class    CG-12520
1       2  CA-2016-152156  11-08-2016  11-11-2016    Second Class    CG-12520
2       3  CA-2016-138688  06-12-2016   6/16/2016    Second Class    DV-13045
3       4  US-2015-108966  10-11-2015  10/18/2015  Standard Class    SO-20335
4       5  US-2015-108966  10-11-2015  10/18/2015  Standard Class    SO-20335

     Customer Name    Segment        Country             City  ...  \
0      Claire Gute   Consumer  United States        Henderson  ...
1      Claire Gute   Consumer  United States        Henderson  ...
2  Darrin Van Huff  Corporate  United States      Los Angeles  ...
3   Sean O'Donnell   Consumer  United States  Fort Lauderdale  ...
4   Sean O'Donnell   Consumer  United States  Fort Lauderdale  ...

  Postal Code  Region       Product ID         Category Sub-Category  \
0       42420   South  FUR-BO-10001798        Furniture    Bookcases
1       42420   South  FUR-CH-10000454        Furniture       Chairs
2       90036    West  OFF-LA-10000240  Office Supplies       Labels
3       33311   South  FUR-TA-10000577        Furniture       Tables
4       33311   South  OFF-ST-10000760  Office Supplies      Storage

                                        Product Name     Sales  Quantity  \
0                 Bush Somerset Collection Bookcase  261.9600         2
1  Hon Deluxe Fabric Upholstered Stacking Chairs,...  731.9400         3
2  Self-Adhesive Address Labels for Typewriters b...   14.6200         2
3      Bretford CR4500 Series Slim Rectangular Table  957.5775         5
4                    Eldon Fold 'N Roll Cart System   22.3680         2

   Discount    Profit
0      0.00   41.9136
1      0.00  219.5820
2      0.00    6.8714
3      0.45 -383.0310
4      0.20    2.5164

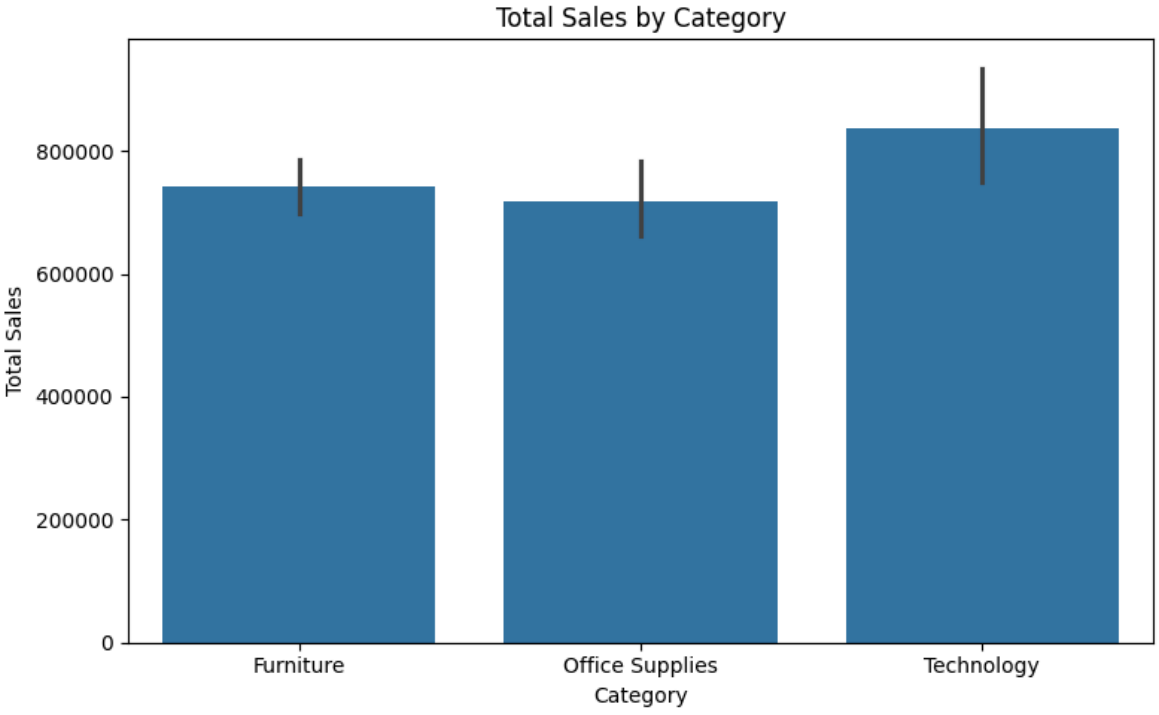
[5 rows x 21 columns]

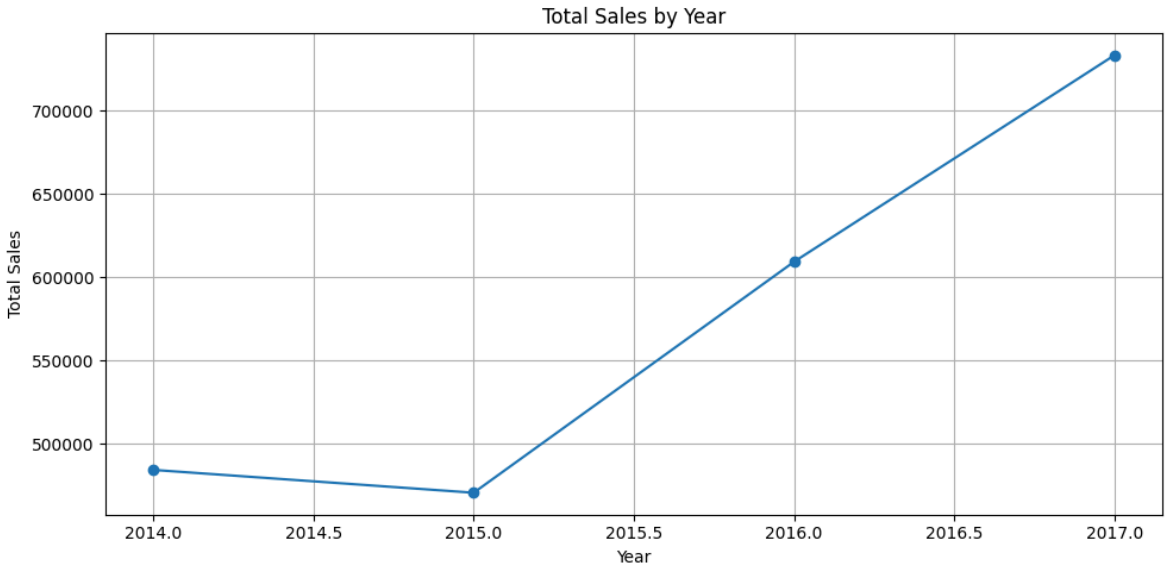
Dataset Shape:
(9994, 21)

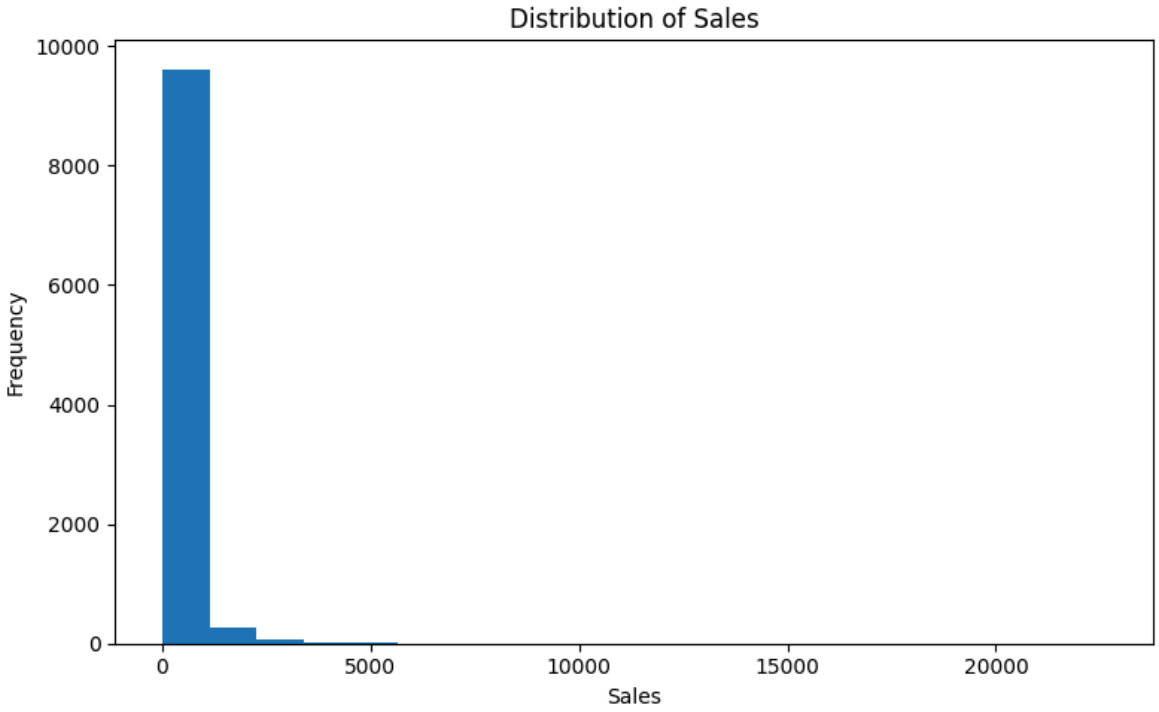
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Row ID         9994 non-null   int64
 1   Order ID       9994 non-null   object
 2   Order Date     9994 non-null   object
 3   Ship Date      9994 non-null   object
 4   Ship Mode      9994 non-null   object
 5   Customer ID    9994 non-null   object
 6   Customer Name  9994 non-null   object
 7   Segment        9994 non-null   object
 8   Country        9994 non-null   object
 9   City           9994 non-null   object
 10  State          9994 non-null   object
```

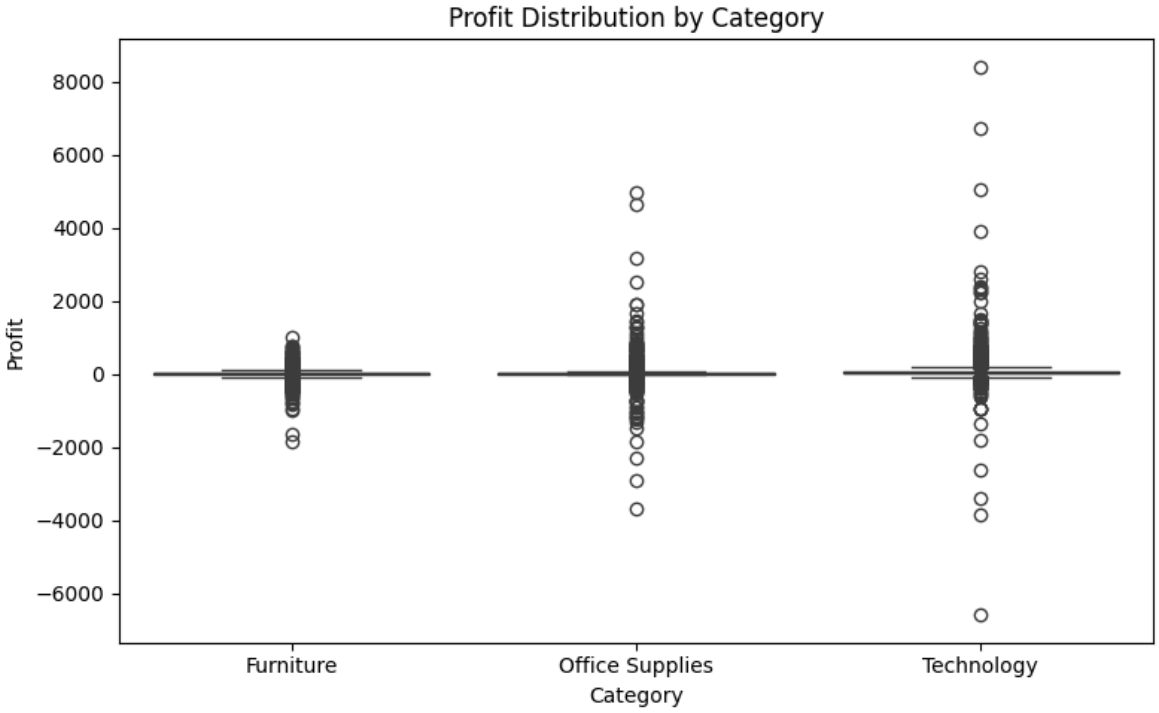
```
 11  Postal Code    9994 non-null   int64
 12  Region         9994 non-null   object
 13  Product ID     9994 non-null   object
 14  Category       9994 non-null   object
 15  Sub-Category   9994 non-null   object
 16  Product Name   9994 non-null   object
 17  Sales          9994 non-null   float64
 18  Quantity       9994 non-null   int64
 19  Discount       9994 non-null   float64
 20  Profit         9994 non-null   float64
dtypes: float64(3), int64(3), object(15)
memory usage: 1.6+ MB

Missing Values:
Row ID           0
Order ID         0
Order Date       0
Ship Date        0
Ship Mode        0
Customer ID      0
Customer Name    0
Segment          0
Country          0
City             0
State            0
Postal Code      0
Region           0
Product ID       0
Category         0
Sub-Category     0
Product Name     0
Sales            0
Quantity         0
Discount         0
Profit           0
dtype: int64
```

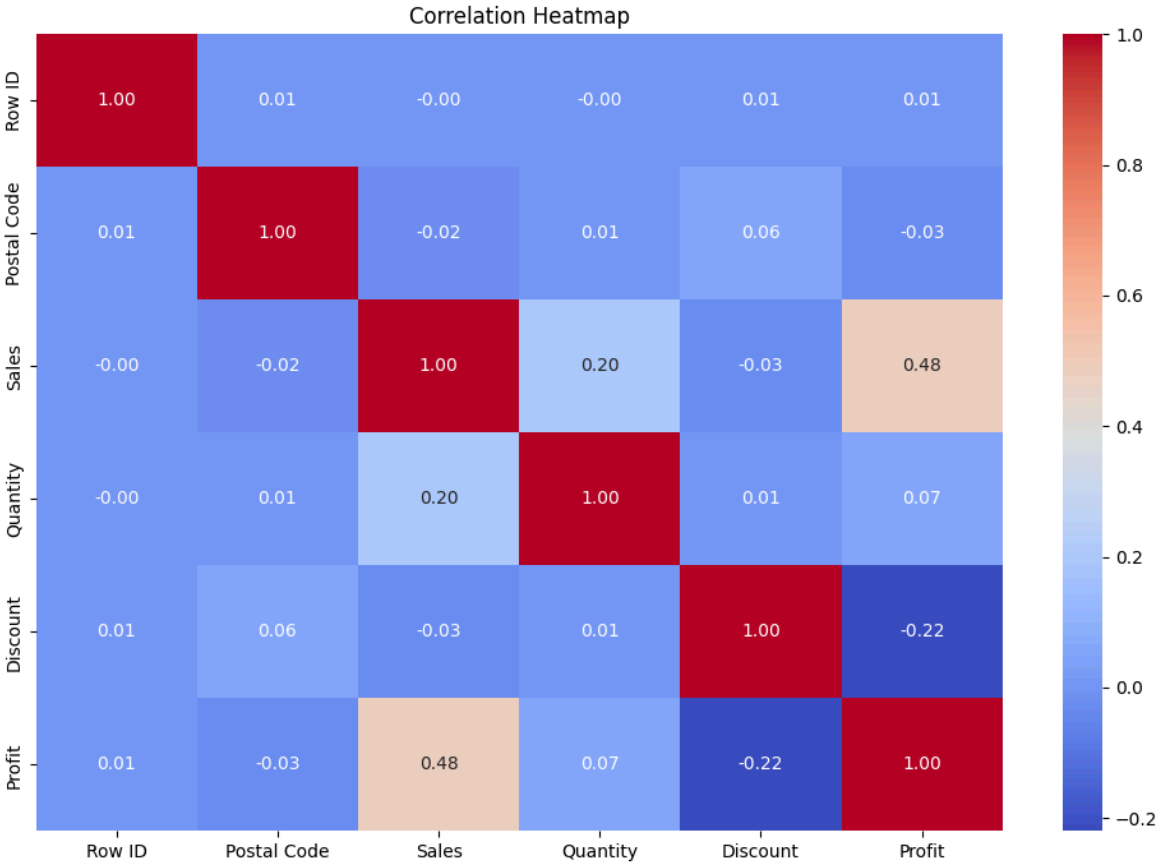


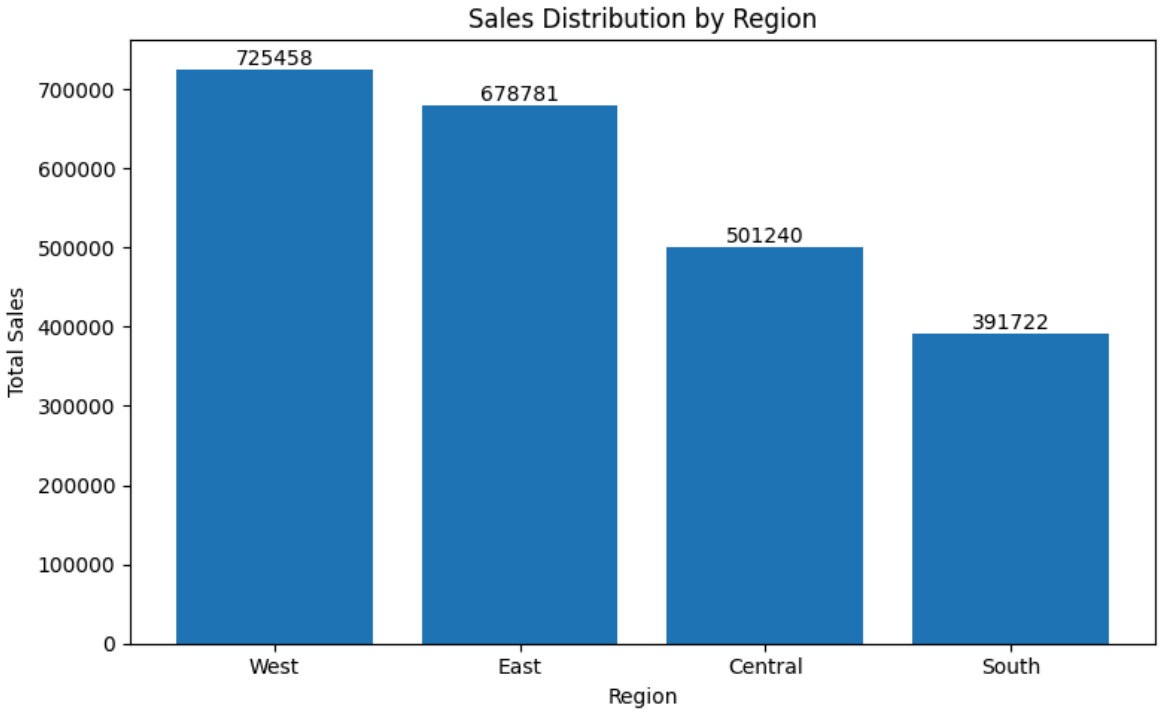

Total Sales by Year
Total Sales
700000
650000
600000
550000
500000
2014.0
2014.5
2015.0
2015.5
2016.0
2016.5
2017.0
Year


Distribution of Sales
Frequency
10000
8000
6000
4000
2000
0
0
5000
10000
15000
20000
Sales


Profit Distribution by Category
Profit
8000
6000
4000
2000
0
−2000
−4000
−6000
Furniture
Office Supplies
Technology
Category


Sales vs Profit
Profit
8000
6000
4000
2000
0
−2000
−4000
−6000
0
5000
10000
15000
20000
Sales
Category
Furniture
Office Supplies
Technology


Correlation Heatmap
Row ID
Postal Code
Sales
Quantity
Discount
Profit
1.00 0.01 -0.00 -0.00 0.01 0.01
0.01 1.00 -0.02 0.01 0.06 -0.03
-0.00 -0.02 1.00 0.20 -0.03 0.48
-0.00 0.01 0.20 1.00 0.01 0.07
0.01 0.06 -0.03 0.01 1.00 -0.22
0.01 -0.03 0.48 0.07 -0.22 1.00
Row ID
Postal Code
Sales
Quantity
Discount
Profit
1.0
0.8
0.6
0.4
0.2
0.0
−0.2


Sales Distribution by Region
Total Sales
0
100000
200000
300000
400000
500000
600000
700000
725458
678781
501240
391722
West
East
Central
South
Region

```
Visualization Summary
----------------------------------------
1. The bar chart shows total sales for each product category.
2. The line chart shows sales trends across different years.
3. The histogram shows the distribution of sales values.
4. The box plot compares profit distribution across categories.
5. The scatter plot shows the relationship between sales and profit.
6. The heatmap shows correlations between numerical variables.
7. The customized chart compares sales across different regions.

Business Insights
----------------------------------------
Total Sales: 2297200.86
Total Profit: 286397.02
Total Orders: 5009
Total Customers: 793
Best Category: Technology
Best Region: West

Experiment completed successfully!
```

Experiment-5

Interactive Dashboard Development using Tableau / Microsoft Power BI

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Step 1: Load Dataset
df = pd.read_csv(
    "Sample - Superstore.csv",
    encoding="latin1"
)
print("Dataset Loaded Successfully")
print(df.head())

# Step 2: Check Dataset
print("\nDataset Shape:")
print(df.shape)
print("\nColumn Names:")
print(df.columns.tolist())

# Step 3: Verify Data
print("\nData Types:")
print(df.dtypes)
print("\nMissing Values:")
print(df.isnull().sum())

# Step 4: Calculate KPIs
total_sales = df["Sales"].sum()
total_profit = df["Profit"].sum()
total_orders = df["Order ID"].nunique()
average_order_value = total_sales / total_orders
print("\n========== KPIs ==========")
print("Total Sales:", round(total_sales, 2))
print("Total Profit:", round(total_profit, 2))
print("Total Orders:", total_orders)
print("Average Order Value:", round(average_order_value, 2))

# Step 5: Bar Chart - Sales by Region
region_sales = df.groupby("Region")["Sales"].sum()
plt.figure(figsize=(8, 5))
region_sales.plot(kind="bar")
plt.title("Sales by Region")
plt.xlabel("Region")
plt.ylabel("Total Sales")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# Step 6: Line Chart - Monthly Sales
df["Order Date"] = pd.to_datetime(
    df["Order Date"],
    format="mixed",
    errors="coerce"
)
monthly_sales = df.groupby(
    df["Order Date"].dt.to_period("M")
```

```
)["Sales"].sum()
plt.figure(figsize=(10, 5))
monthly_sales.plot(
    kind="line",
    marker="o"
)
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Step 7: Pie Chart - Sales by Category
category_sales = df.groupby("Category")["Sales"].sum()
plt.figure(figsize=(7, 7))
category_sales.plot(
    kind="pie",
    autopct="%1.1f%%"
)
plt.title("Sales by Category")
plt.ylabel("")
plt.tight_layout()
plt.show()

# Step 8: Correlation Heatmap
numeric_data = df.select_dtypes(
    include="number"
)
correlation = numeric_data.corr()
plt.figure(figsize=(9, 6))
sns.heatmap(
    correlation,
    annot=True,
    fmt=".2f",
    cmap="coolwarm"
)
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()

# Step 9: Filter / Slicer
print("\nAvailable Regions:")
print(df["Region"].unique())
region = input(
    "\nEnter Region (East/West/North/South): "
)
filtered_data = df[
    df["Region"].str.lower() == region.lower()
]
print("\n========== Filtered Data ==========")
print(filtered_data.head())
filtered_sales = filtered_data["Sales"].sum()
filtered_profit = filtered_data["Profit"].sum()
print("\nFiltered Sales:",
      round(filtered_sales, 2))
print("Filtered Profit:",
      round(filtered_profit, 2))

# Step 10: Business Insights
```

```
print("\n========== Business Insights ==========")
top_region = region_sales.idxmax()
top_category = category_sales.idxmax()
print("Top Sales Region:", top_region)
print(
    "Highest Region Sales:",
    round(region_sales.max(), 2)
)
print("Top Sales Category:", top_category)
print(
    "Highest Category Sales:",
    round(category_sales.max(), 2)
)
```

```
Dataset Loaded Successfully
   Row ID        Order ID  Order Date   Ship Date       Ship Mode Customer ID  \
0       1  CA-2016-152156  11-08-2016  11-11-2016    Second Class    CG-12520
1       2  CA-2016-152156  11-08-2016  11-11-2016    Second Class    CG-12520
2       3  CA-2016-138688  06-12-2016   6/16/2016    Second Class    DV-13045
3       4  US-2015-108966  10-11-2015  10/18/2015  Standard Class    SO-20335
4       5  US-2015-108966  10-11-2015  10/18/2015  Standard Class    SO-20335

     Customer Name    Segment        Country             City  ...  \
0      Claire Gute   Consumer  United States        Henderson  ...
1      Claire Gute   Consumer  United States        Henderson  ...
2  Darrin Van Huff  Corporate  United States      Los Angeles  ...
3   Sean O'Donnell   Consumer  United States  Fort Lauderdale  ...
4   Sean O'Donnell   Consumer  United States  Fort Lauderdale  ...

  Postal Code  Region       Product ID         Category Sub-Category  \
0       42420   South  FUR-BO-10001798        Furniture    Bookcases
1       42420   South  FUR-CH-10000454        Furniture       Chairs
2       90036    West  OFF-LA-10000240  Office Supplies       Labels
3       33311   South  FUR-TA-10000577        Furniture       Tables
4       33311   South  OFF-ST-10000760  Office Supplies      Storage

                                        Product Name     Sales  Quantity  \
0                 Bush Somerset Collection Bookcase  261.9600         2
1  Hon Deluxe Fabric Upholstered Stacking Chairs,...  731.9400         3
2  Self-Adhesive Address Labels for Typewriters b...   14.6200         2
3      Bretford CR4500 Series Slim Rectangular Table  957.5775         5
4                    Eldon Fold 'N Roll Cart System   22.3680         2

   Discount    Profit
0      0.00   41.9136
1      0.00  219.5820
2      0.00    6.8714
3      0.45 -383.0310
4      0.20    2.5164

[5 rows x 21 columns]

Dataset Shape:
(9994, 21)

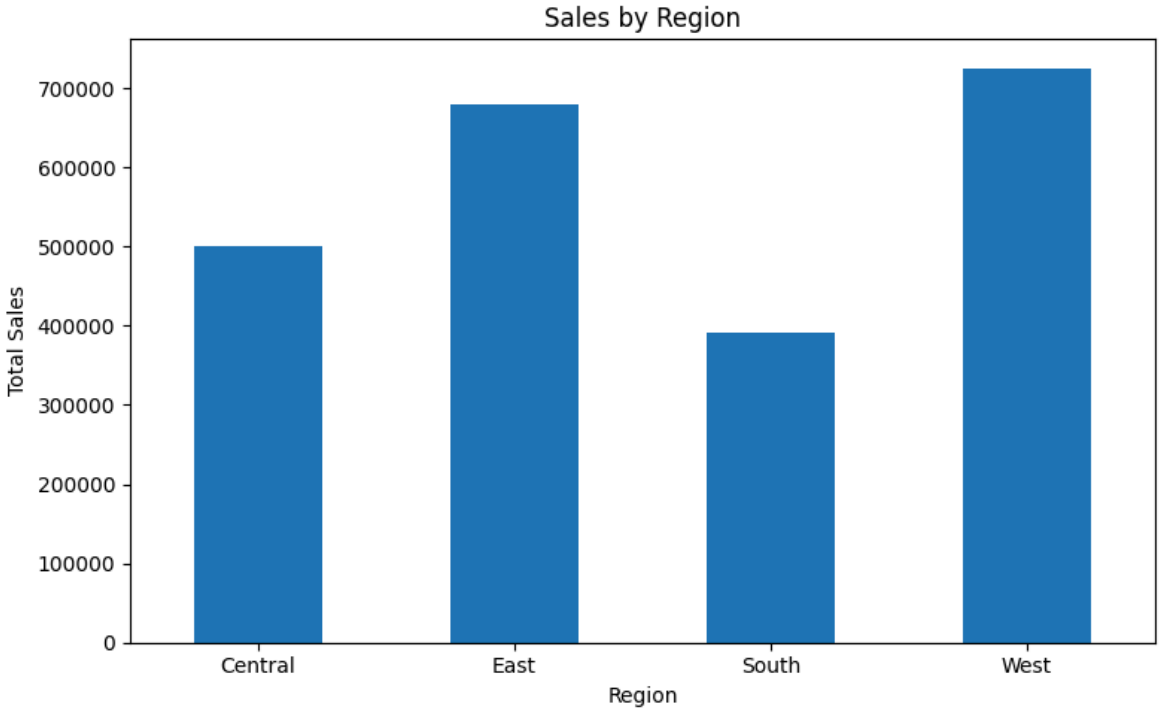
Column Names:
['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID', 'Cu
stomer Name', 'Segment', 'Country', 'City', 'State', 'Postal Code', 'Region', 'Pr
oduct ID', 'Category', 'Sub-Category', 'Product Name', 'Sales', 'Quantity', 'Disc
ount', 'Profit']

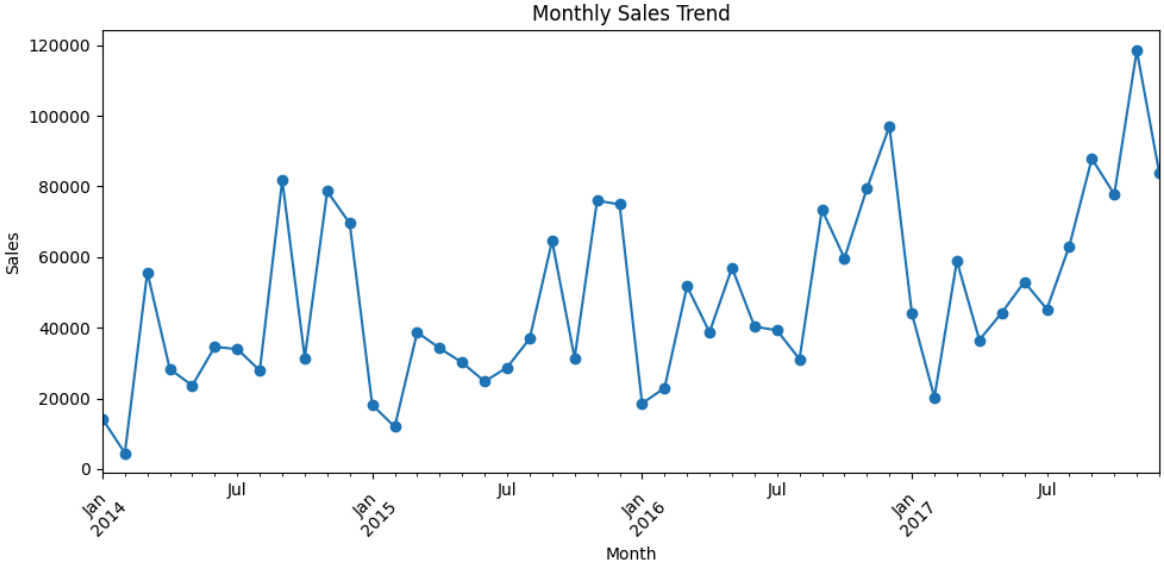
Data Types:
Row ID            int64
Order ID         object
Order Date       object
Ship Date        object
Ship Mode        object
Customer ID      object
Customer Name    object
Segment          object
Country          object
City             object
State            object
Postal Code       int64
```

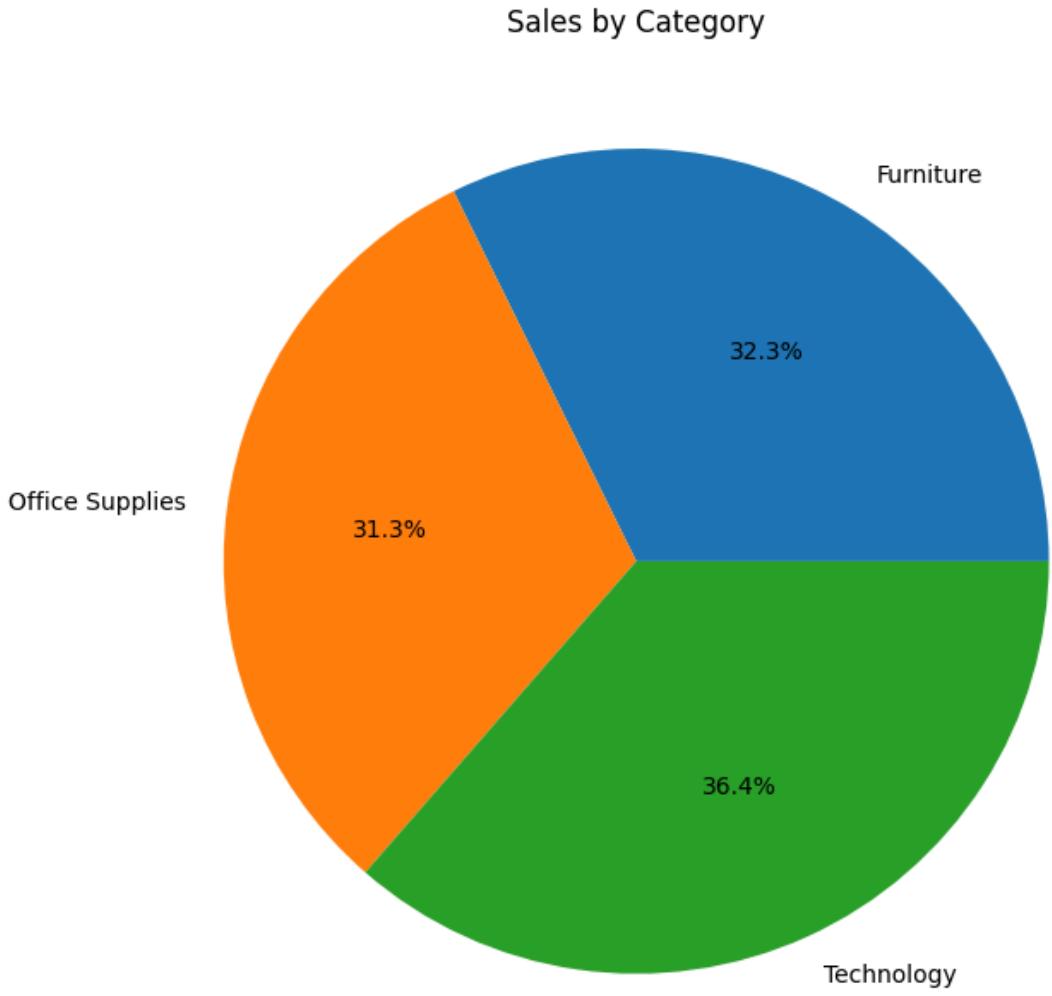
```
Region            object
Product ID        object
Category          object
Sub-Category      object
Product Name      object
Sales            float64
Quantity           int64
Discount         float64
Profit           float64
dtype: object

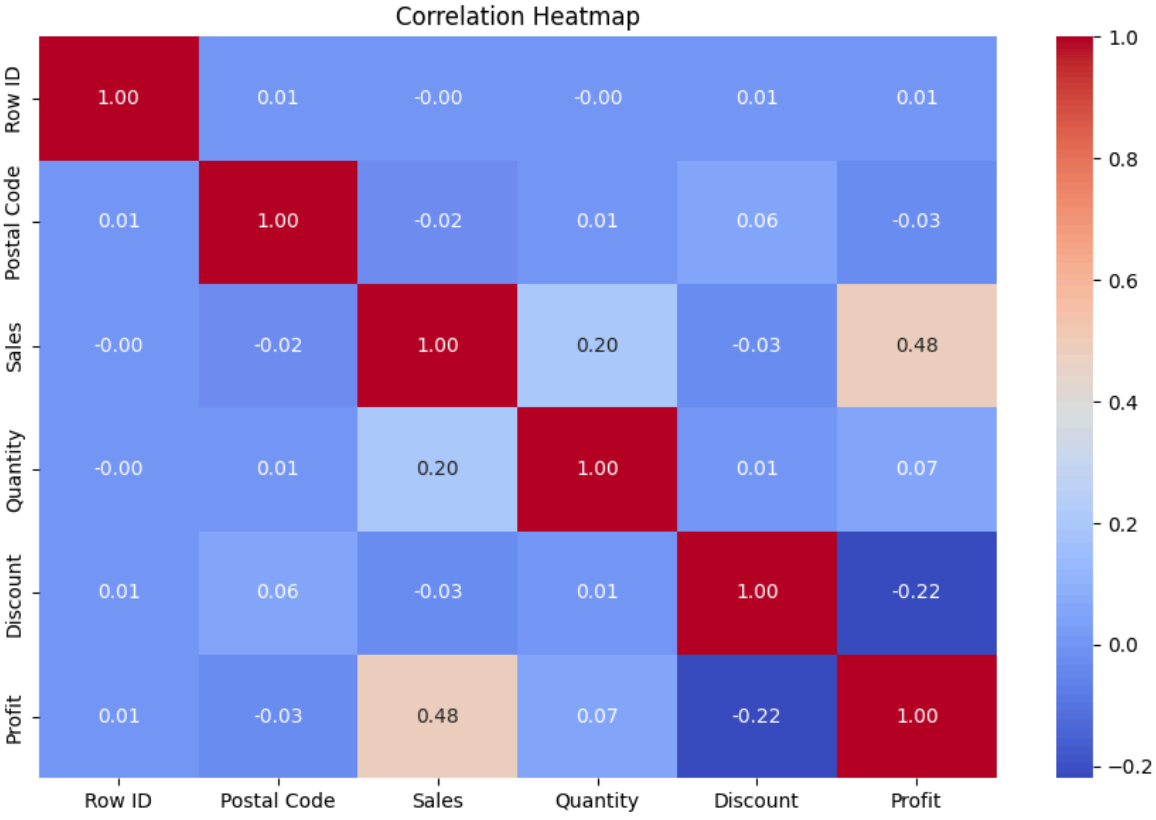
Missing Values:
Row ID           0
Order ID         0
Order Date       0
Ship Date        0
Ship Mode        0
Customer ID      0
Customer Name    0
Segment          0
Country          0
City             0
State            0
Postal Code      0
Region           0
Product ID       0
Category         0
Sub-Category     0
Product Name     0
Sales            0
Quantity         0
Discount         0
Profit           0
dtype: int64

========== KPIs ==========
Total Sales: 2297200.86
Total Profit: 286397.02
Total Orders: 5009
Average Order Value: 458.61
```


Sales by Region
Total Sales
0
100000
200000
300000
400000
500000
600000
700000
Central
East
South
West
Region


Monthly Sales Trend
Sales
0
20000
40000
60000
80000
100000
120000
Jan 2014
Jul
Jan 2015
Jul
Jan 2016
Jul
Jan 2017
Jul
Month


Sales by Category
Furniture
32.3%
Office Supplies
31.3%
36.4%
Technology


Correlation Heatmap
Row ID
Postal Code
Sales
Quantity
Discount
Profit
1.00 0.01 -0.00 -0.00 0.01 0.01
0.01 1.00 -0.02 0.01 0.06 -0.03
-0.00 -0.02 1.00 0.20 -0.03 0.48
-0.00 0.01 0.20 1.00 0.01 0.07
0.01 0.06 -0.03 0.01 1.00 -0.22
0.01 -0.03 0.48 0.07 -0.22 1.00
1.0
0.8
0.6
0.4
0.2
0.0
−0.2

```
Available Regions:
['South' 'West' 'Central' 'East']

========== Filtered Data ==========
Empty DataFrame
Columns: [Row ID, Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Custom
er Name, Segment, Country, City, State, Postal Code, Region, Product ID, Categor
y, Sub-Category, Product Name, Sales, Quantity, Discount, Profit]
Index: []

[0 rows x 21 columns]

Filtered Sales: 0.0
Filtered Profit: 0.0

========== Business Insights ==========
Top Sales Region: West
Highest Region Sales: 725457.82
Top Sales Category: Technology
Highest Category Sales: 836154.03
```

EXPERIMENT NO. 6

Predictive Analytics using Linear Regression and

Model Performance Evaluation

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score


# Step 1: Select and load the dataset
housing = fetch_california_housing()

df = pd.DataFrame(
    housing.data,
    columns=housing.feature_names
)

df["MedHouseVal"] = housing.target

print("Step 1: Dataset Loaded Successfully")
print(df.head())


# Step 2: Import libraries and check dataset
print("\nStep 2: Dataset Information")
print("Shape:", df.shape)
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())


# Step 3: Data preprocessing and feature selection
X = df.drop("MedHouseVal", axis=1)
y = df["MedHouseVal"]

print("\nStep 3: Features and Target Selected")
print("Features:", X.columns.tolist())
print("Target: MedHouseVal")


# Step 4: Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("\nStep 4: Dataset Split")
print("Training Data:", X_train.shape)
```

```
print("Testing Data:", X_test.shape)


# Step 5: Train Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

print("\nStep 5: Linear Regression Model Trained Successfully")


# Step 6: Predict values using the trained model
y_pred = model.predict(X_test)

print("\nStep 6: Prediction Completed")
print("First 10 Predicted Values:")
print(y_pred[:10])


# Step 7: Visualize actual and predicted values
plt.figure(figsize=(8, 6))

plt.scatter(y_test, y_pred, alpha=0.5)

min_value = min(y_test.min(), y_pred.min())
max_value = max(y_test.max(), y_pred.max())

plt.plot(
    [min_value, max_value],
    [min_value, max_value],
    linestyle="--"
)

plt.xlabel("Actual House Values")
plt.ylabel("Predicted House Values")
plt.title("Actual vs Predicted House Values")
plt.show()


# Step 8: Evaluate the model
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nStep 8: Model Performance Evaluation")
print("Mean Absolute Error (MAE):", round(mae, 4))
print("Mean Squared Error (MSE):", round(mse, 4))
print("Root Mean Squared Error (RMSE):", round(rmse, 4))
print("R² Score:", round(r2, 4))


# Step 9: Analyze regression coefficients
coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
})

print("\nStep 9: Regression Coefficients")
print(coefficients)
```

```
plt.figure(figsize=(10, 6))

plt.barh(
    coefficients["Feature"],
    coefficients["Coefficient"]
)

plt.xlabel("Coefficient Value")
plt.ylabel("Features")
plt.title("Regression Coefficients")
plt.show()


# Step 10: Interpret and display final result
comparison = pd.DataFrame({
    "Actual Value": y_test.values,
    "Predicted Value": y_pred
})

print("\nStep 10: Actual vs Predicted Values")
print(comparison.head(10))
```

```
Step 1: Dataset Loaded Successfully
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85

   Longitude  MedHouseVal
0    -122.23        4.526
1    -122.22        3.585
2    -122.24        3.521
3    -122.25        3.413
4    -122.25        3.422

Step 2: Dataset Information
Shape: (20640, 9)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   MedInc       20640 non-null  float64
 1   HouseAge     20640 non-null  float64
 2   AveRooms     20640 non-null  float64
 3   AveBedrms    20640 non-null  float64
 4   Population   20640 non-null  float64
 5   AveOccup     20640 non-null  float64
 6   Latitude     20640 non-null  float64
 7   Longitude    20640 non-null  float64
 8   MedHouseVal  20640 non-null  float64
dtypes: float64(9)
memory usage: 1.4 MB
None

Missing Values:
MedInc         0
HouseAge       0
AveRooms       0
AveBedrms      0
Population     0
AveOccup       0
Latitude       0
Longitude      0
MedHouseVal    0
dtype: int64

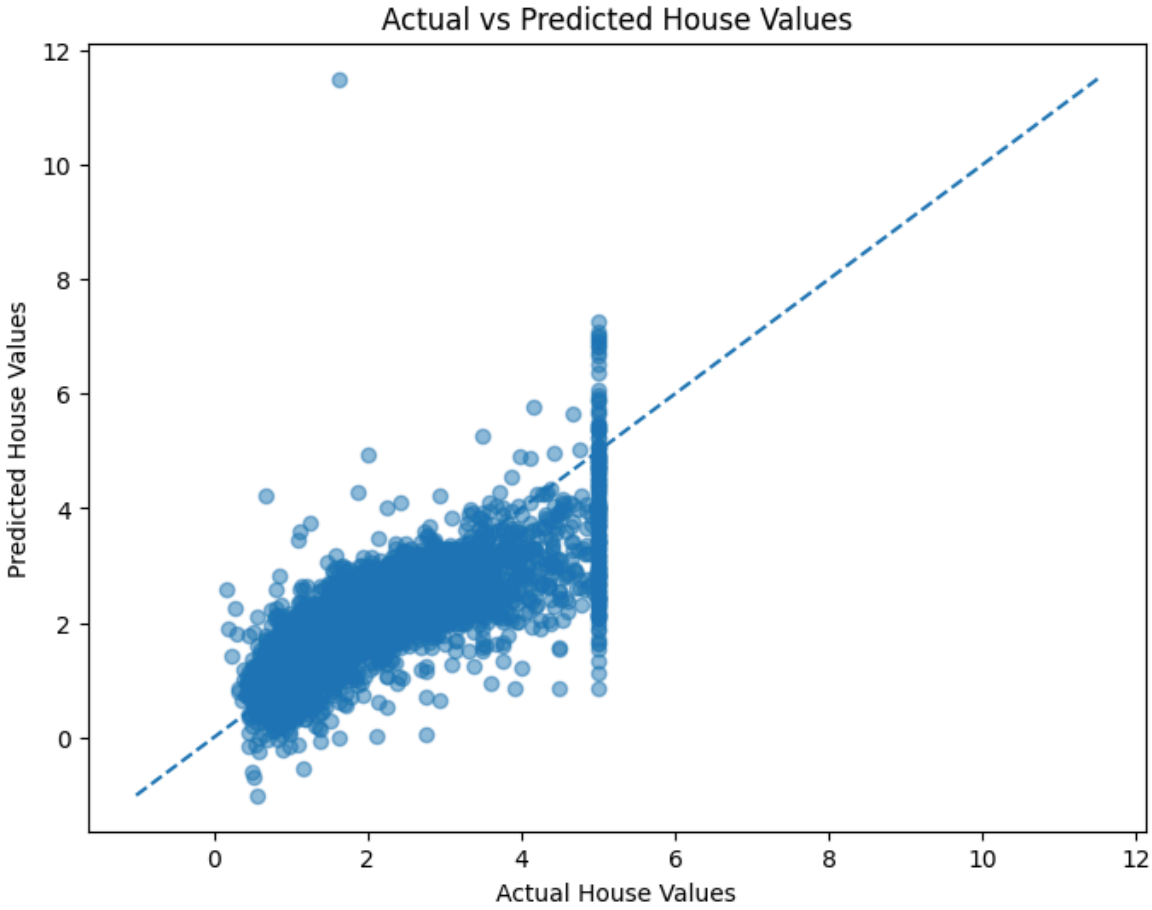
Step 3: Features and Target Selected
Features: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccu
p', 'Latitude', 'Longitude']
Target: MedHouseVal

Step 4: Dataset Split
Training Data: (16512, 8)
Testing Data: (4128, 8)

Step 5: Linear Regression Model Trained Successfully

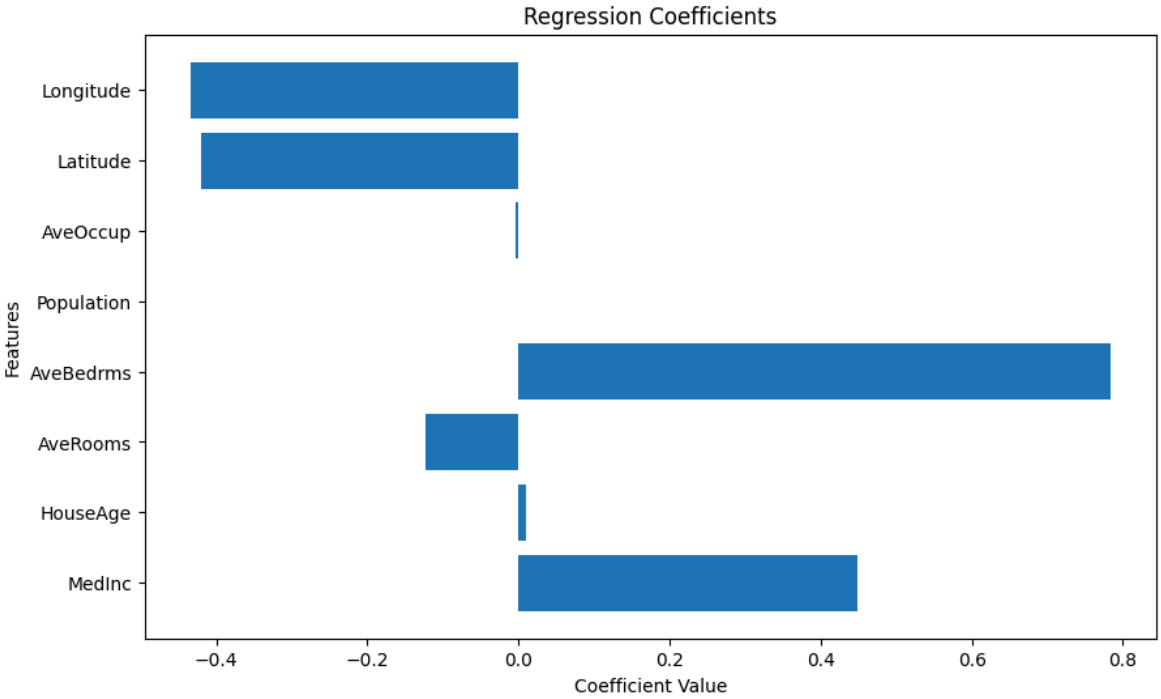
Step 6: Prediction Completed
First 10 Predicted Values:
```

```
[0.71912284 1.76401657 2.70965883 2.83892593 2.60465725 2.01175367
 2.64550005 2.16875532 2.74074644 3.91561473]
```




```
Step 8: Model Performance Evaluation
Mean Absolute Error (MAE): 0.5332
Mean Squared Error (MSE): 0.5559
Root Mean Squared Error (RMSE): 0.7456
R² Score: 0.5758

Step 9: Regression Coefficients
      Feature  Coefficient
0      MedInc     0.448675
1    HouseAge     0.009724
2    AveRooms    -0.123323
3   AveBedrms     0.783145
4  Population    -0.000002
5    AveOccup    -0.003526
6    Latitude    -0.419792
7   Longitude    -0.433708
```



Step 10: Actual vs Predicted Values

	Actual Value	Predicted Value
0	0.47700	0.719123
1	0.45800	1.764017
2	5.00001	2.709659
3	2.18600	2.838926
4	2.78000	2.604657
5	1.58700	2.011754
6	1.98200	2.645500
7	1.57500	2.168755
8	3.40000	2.740746
9	4.46600	3.915615

Experiment - 7

Customer Segmentation using K-Means Clustering

In [1]:
```
# Step 1: Load Dataset

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

df = pd.read_csv("Mall_Customers.csv")

print(df.head())


# Step 2: Explore Dataset

print("Dataset Shape:")
print(df.shape)

print("\nDataset Information:")
print(df.info())

print("\nStatistical Summary:")
print(df.describe())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Values:")
print(df.duplicated().sum())


# Step 3: Data Preprocessing

df = df.drop_duplicates()
df = df.dropna()

print("\nDataset after preprocessing:")
print(df.shape)


# Step 4: Select Relevant Features

X = df[[
    "Age",
    "Annual Income (k$)",
    "Spending Score (1-100)"
]]

print("\nSelected Features:")
print(X.head())
```

```
# Step 5: Feature Scaling

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print("\nScaled Data:")
print(X_scaled[:5])


# Step 6: Elbow Method

inertia = []

for k in range(2, 11):

    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    model.fit(X_scaled)

    inertia.append(model.inertia_)

plt.figure(figsize=(8, 5))

plt.plot(
    range(2, 11),
    inertia,
    marker="o"
)

plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")
plt.grid(True)

plt.show()


# Step 7: Silhouette Score

for k in range(2, 11):

    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    labels = model.fit_predict(X_scaled)

    score = silhouette_score(
        X_scaled,
        labels
    )
```

```
    print(
        "K =", k,
        "Silhouette Score =", round(score, 4)
    )


# Step 8: Apply K-Means Clustering

k = 5

kmeans = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

df["Cluster"] = kmeans.fit_predict(X_scaled)

print("\nClustered Data:")
print(df.head())


# Step 9: Calculate Cluster Centroids

centroids = scaler.inverse_transform(
    kmeans.cluster_centers_
)

centroids_df = pd.DataFrame(
    centroids,
    columns=[
        "Age",
        "Annual Income (k$)",
        "Spending Score (1-100)"
    ]
)

print("\nCluster Centroids:")
print(centroids_df)


# Step 10: Visualize Customer Segments

plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x="Annual Income (k$)",
    y="Spending Score (1-100)",
    hue="Cluster",
    s=100
)

plt.scatter(
    centroids_df["Annual Income (k$)"],
    centroids_df["Spending Score (1-100)"],
    marker="X",
    s=250,
    c="black",
    label="Centroids"
```

```
)

plt.title("Customer Segmentation using K-Means")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score")
plt.legend()

plt.show()


# Step 11: Analyze Customer Clusters

cluster_analysis = df.groupby("Cluster")[
    [
        "Age",
        "Annual Income (k$)",
        "Spending Score (1-100)"
    ]
].mean()

print("\nCluster Analysis:")
print(cluster_analysis.round(2))


# Step 12: Count Customers in Each Cluster

print("\nNumber of Customers in Each Cluster:")

print(
    df["Cluster"].value_counts().sort_index()
)


# Step 13: Save the Final Dataset

df.to_csv(
    "customer_segments.csv",
    index=False
)

print("\nCustomer segmentation completed successfully.")
```

```
   CustomerID   Genre  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40
Dataset Shape:
(200, 5)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Genre                   200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
None

Statistical Summary:
       CustomerID         Age  Annual Income (k$)  Spending Score (1-100)
count  200.000000  200.000000          200.000000              200.000000
mean   100.500000   38.850000           60.560000               50.200000
std     57.879185   13.969007           26.264721               25.823522
min      1.000000   18.000000           15.000000                1.000000
25%     50.750000   28.750000           41.500000               34.750000
50%    100.500000   36.000000           61.500000               50.000000
75%    150.250000   49.000000           78.000000               73.000000
max    200.000000   70.000000          137.000000               99.000000

Missing Values:
CustomerID                0
Genre                     0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64

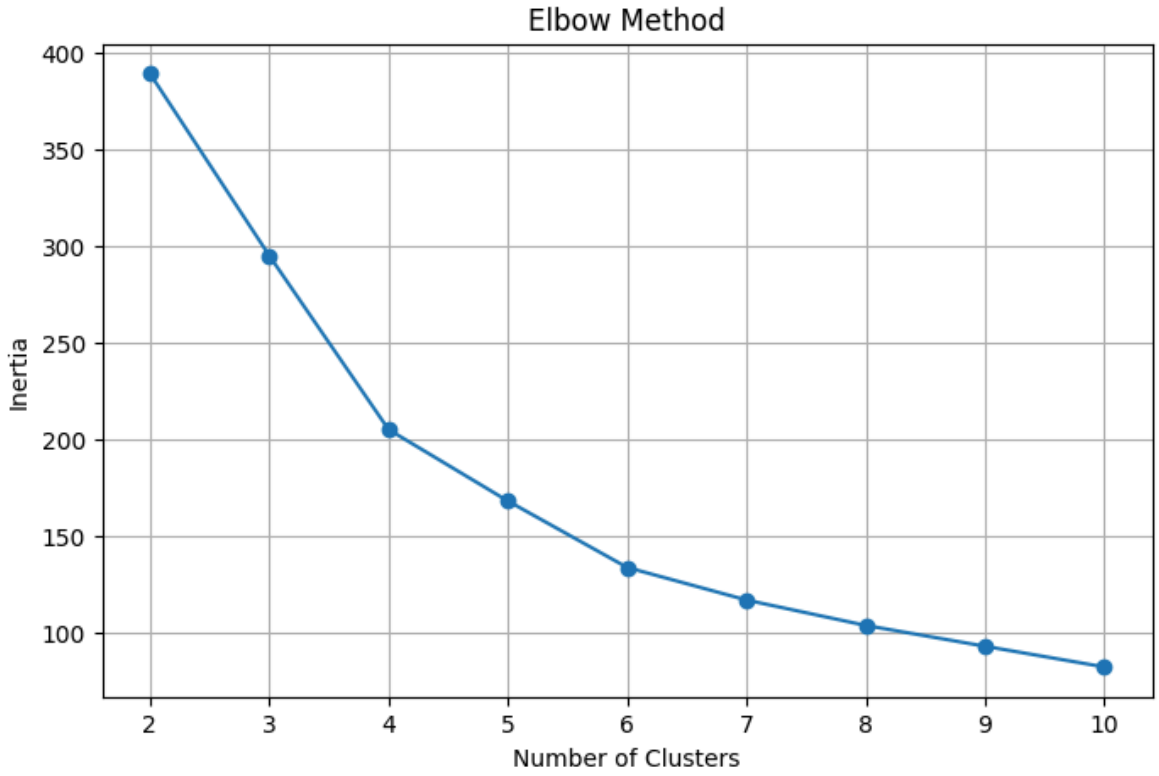
Duplicate Values:
0

Dataset after preprocessing:
(200, 5)

Selected Features:
   Age  Annual Income (k$)  Spending Score (1-100)
0   19                  15                      39
1   21                  15                      81
2   20                  16                       6
3   23                  16                      77
4   31                  17                      40

Scaled Data:
[[-1.42456879 -1.73899919 -0.43480148]
 [-1.28103541 -1.73899919  1.19570407]
```

```
[-1.3528021  -1.70082976 -1.71591298]
[-1.13750203 -1.70082976  1.04041783]
[-0.56336851 -1.66266033 -0.39597992]]
```




```
K = 2 Silhouette Score = 0.3355
K = 3 Silhouette Score = 0.3578
K = 4 Silhouette Score = 0.404
K = 5 Silhouette Score = 0.4166
K = 6 Silhouette Score = 0.4284
K = 7 Silhouette Score = 0.4172
K = 8 Silhouette Score = 0.4082
K = 9 Silhouette Score = 0.4177
K = 10 Silhouette Score = 0.4066

Clustered Data:
   CustomerID   Genre  Age  Annual Income (k$)  Spending Score (1-100)  \
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

   Cluster
0        1
1        1
2        0
3        1
4        1

Cluster Centroids:
         Age  Annual Income (k$)  Spending Score (1-100)
0  46.250000           26.750000               18.350000
1  25.185185           41.092593               62.240741
2  32.875000           86.100000               81.525000
3  39.871795           86.102564               19.358974
4  55.638298           54.382979               48.851064
```




```
Cluster Analysis:
           Age  Annual Income (k$)  Spending Score (1-100)
Cluster
0        46.25               26.75                   18.35
1        25.19               41.09                   62.24
2        32.88               86.10                   81.53
3        39.87               86.10                   19.36
4        55.64               54.38                   48.85

Number of Customers in Each Cluster:
Cluster
0    20
1    54
2    40
3    39
4    47
Name: count, dtype: int64

Customer segmentation completed successfully.
```

Experiment-8

Customer Churn Prediction using Decision Tree Classification

In [1]:
```
# Step 1: Load Dataset

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
from sklearn.tree import plot_tree

df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")

print("First 5 Records:")
print(df.head())


# Step 2: Explore Dataset

print("\nDataset Shape:")
print(df.shape)

print("\nDataset Information:")
print(df.info())

print("\nStatistical Summary:")
print(df.describe())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Values:")
print(df.duplicated().sum())


# Step 3: Data Preprocessing

df = df.drop_duplicates()

df["TotalCharges"] = pd.to_numeric(
    df["TotalCharges"],
    errors="coerce"
)

df["TotalCharges"] = df["TotalCharges"].fillna(
```

```
    df["TotalCharges"].median()
)

print("\nMissing Values After Preprocessing:")
print(df.isnull().sum())


# Step 4: Remove Unnecessary Columns

df = df.drop(
    columns=["customerID"]
)

print("\nColumns After Removing Customer ID:")
print(df.columns)


# Step 5: Encode Categorical Variables

categorical_columns = df.select_dtypes(
    include=["object"]
).columns

encoder = LabelEncoder()

for column in categorical_columns:
    df[column] = encoder.fit_transform(
        df[column]
    )

print("\nEncoded Dataset:")
print(df.head())


# Step 6: Select Features and Target

X = df.drop(
    columns=["Churn"]
)

y = df["Churn"]

print("\nFeatures:")
print(X.columns)

print("\nTarget:")
print(y.name)


# Step 7: Split Dataset into Training and Testing Data

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining Data Shape:")
```

```
print(X_train.shape)

print("\nTesting Data Shape:")
print(X_test.shape)


# Step 8: Train Decision Tree Classifier

model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=5,
    random_state=42
)

model.fit(
    X_train,
    y_train
)

print("\nDecision Tree Model Trained Successfully.")


# Step 9: Predict Customer Churn

y_pred = model.predict(
    X_test
)

print("\nPredicted Values:")
print(y_pred[:20])


# Step 10: Evaluate Model

accuracy = accuracy_score(
    y_test,
    y_pred
)

precision = precision_score(
    y_test,
    y_pred
)

recall = recall_score(
    y_test,
    y_pred
)

f1 = f1_score(
    y_test,
    y_pred
)

print("\nModel Evaluation:")

print("Accuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1 Score :", round(f1, 4))
```

```
# Step 11: Classification Report

print("\nClassification Report:")

print(
    classification_report(
        y_test,
        y_pred
    )
)


# Step 12: Confusion Matrix

cm = confusion_matrix(
    y_test,
    y_pred
)

print("\nConfusion Matrix:")
print(cm)

plt.figure(figsize=(7, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["No Churn", "Churn"],
    yticklabels=["No Churn", "Churn"]
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()


# Step 13: Visualize Decision Tree

plt.figure(figsize=(20, 10))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=["No Churn", "Churn"],
    filled=True,
    rounded=True,
    fontsize=8
)

plt.title("Decision Tree for Customer Churn Prediction")

plt.show()
```

```
# Step 14: Find Feature Importance

feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)

print("\nFeature Importance:")
print(feature_importance)


# Step 15: Visualize Feature Importance

plt.figure(figsize=(10, 6))

sns.barplot(
    data=feature_importance,
    x="Importance",
    y="Feature"
)

plt.title("Feature Importance in Customer Churn Prediction")
plt.xlabel("Importance")
plt.ylabel("Feature")

plt.show()


# Step 16: Analyze Churn Distribution

churn_count = df["Churn"].value_counts()

print("\nCustomer Churn Distribution:")
print(churn_count)

plt.figure(figsize=(6, 5))

sns.countplot(
    x=df["Churn"]
)

plt.title("Customer Churn Distribution")
plt.xlabel("Churn")
plt.ylabel("Number of Customers")

plt.show()


# Step 17: Display Customers Predicted to Churn

churn_predictions = X_test.copy()

churn_predictions["Actual Churn"] = y_test.values
churn_predictions["Predicted Churn"] = y_pred
```

```
print("\nCustomers Predicted to Churn:")

print(
    churn_predictions[
        churn_predictions["Predicted Churn"] == 1
    ].head(20)
)


# Step 18: Final Result

print("\nFinal Result:")

print(
    "Decision Tree Customer Churn Prediction completed successfully."
)

print(
    f"Model Accuracy: {accuracy * 100:.2f}%"
)

print(
    f"Precision: {precision * 100:.2f}%"
)

print(
    f"Recall: {recall * 100:.2f}%"
)

print(
    f"F1 Score: {f1 * 100:.2f}%"
)
```

```
First 5 Records:
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes

      MultipleLines InternetService OnlineSecurity  ... DeviceProtection  \
0  No phone service             DSL             No  ...               No
1                No             DSL            Yes  ...              Yes
2                No             DSL            Yes  ...               No
3  No phone service             DSL            Yes  ...              Yes
4                No     Fiber optic             No  ...               No

  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year               No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year               No
4          No          No              No  Month-to-month              Yes

               PaymentMethod MonthlyCharges  TotalCharges Churn
0           Electronic check          29.85         29.85    No
1               Mailed check          56.95        1889.5    No
2               Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4           Electronic check          70.70        151.65   Yes

[5 rows x 21 columns]

Dataset Shape:
(7043, 21)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
```

```
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
None

Statistical Summary:
       SeniorCitizen       tenure  MonthlyCharges
count    7043.000000  7043.000000     7043.000000
mean        0.162147    32.371149       64.761692
std         0.368612    24.559481       30.090047
min         0.000000     0.000000       18.250000
25%         0.000000     9.000000       35.500000
50%         0.000000    29.000000       70.350000
75%         0.000000    55.000000       89.850000
max         1.000000    72.000000      118.750000

Missing Values:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

Duplicate Values:
0

Missing Values After Preprocessing:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
```

```
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

Columns After Removing Customer ID:
Index(['gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure',
       'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity',
       'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV',
       'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod',
       'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='object')

Encoded Dataset:
   gender  SeniorCitizen  Partner  Dependents  tenure  PhoneService  \
0       0              0        1           0       1             0
1       1              0        0           0      34             1
2       1              0        0           0       2             1
3       1              0        0           0      45             0
4       0              0        0           0       2             1

   MultipleLines  InternetService  OnlineSecurity  OnlineBackup  \
0              1                0               0             2
1              0                0               2             0
2              0                0               2             2
3              1                0               2             0
4              0                1               0             0

   DeviceProtection  TechSupport  StreamingTV  StreamingMovies  Contract  \
0                 0            0            0                0         0
1                 2            0            0                0         1
2                 0            0            0                0         0
3                 2            2            0                0         1
4                 0            0            0                0         0

   PaperlessBilling  PaymentMethod  MonthlyCharges  TotalCharges  Churn
0                 1              2           29.85         29.85      0
1                 0              3           56.95       1889.50      0
2                 1              3           53.85        108.15      1
3                 0              0           42.30       1840.75      0
4                 1              2           70.70        151.65      1

Features:
Index(['gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure',
       'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity',
       'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV',
       'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod',
       'MonthlyCharges', 'TotalCharges'],
      dtype='object')

Target:
Churn

Training Data Shape:
(5634, 19)

Testing Data Shape:
(1409, 19)
```

```
Decision Tree Model Trained Successfully.

Predicted Values:
[0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0]

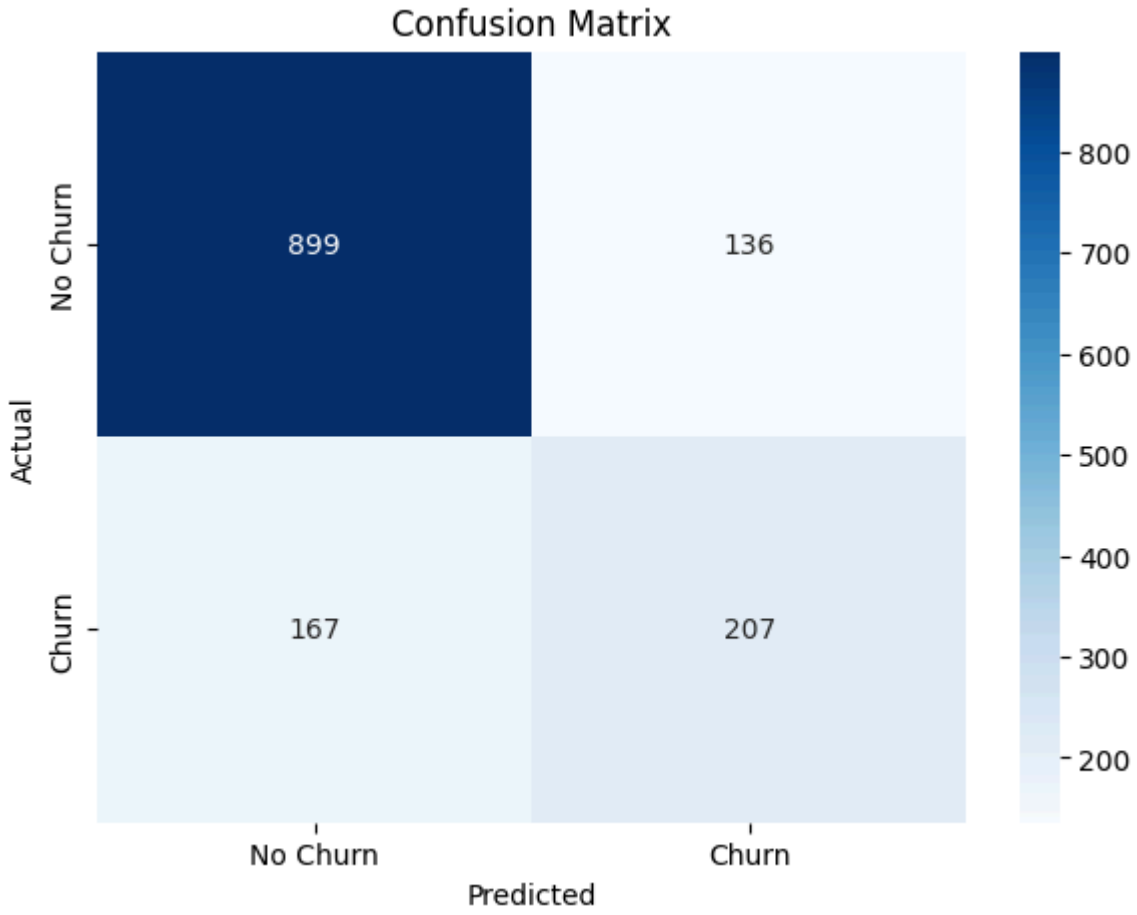
Model Evaluation:
Accuracy : 0.785
Precision: 0.6035
Recall   : 0.5535
F1 Score : 0.5774

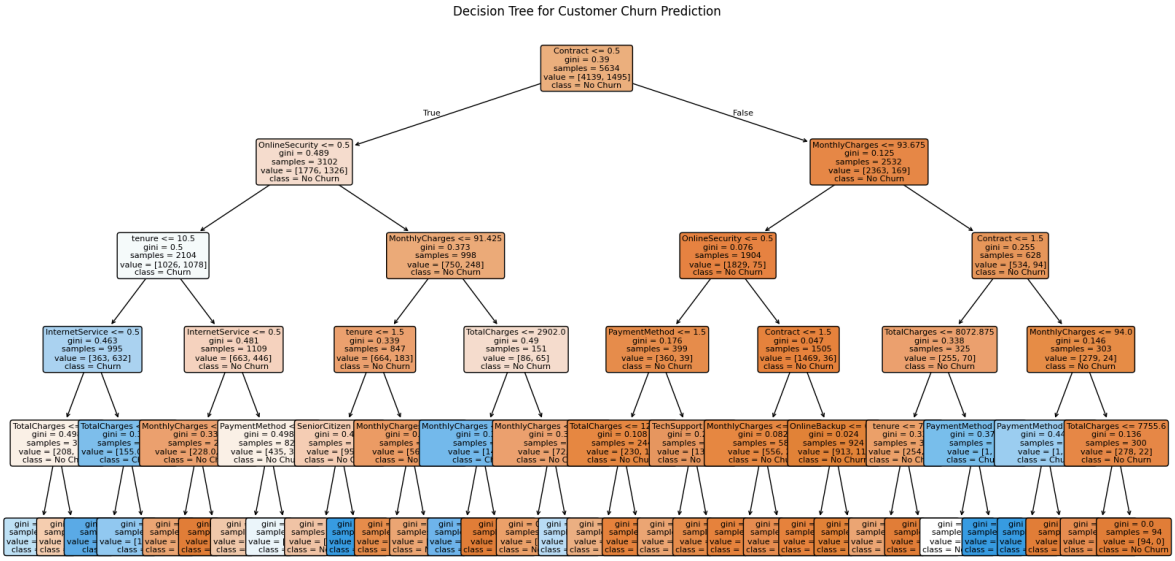
Classification Report:
              precision    recall  f1-score   support

           0       0.84      0.87      0.86      1035
           1       0.60      0.55      0.58       374

    accuracy                           0.78      1409
   macro avg       0.72      0.71      0.72      1409
weighted avg       0.78      0.78      0.78      1409

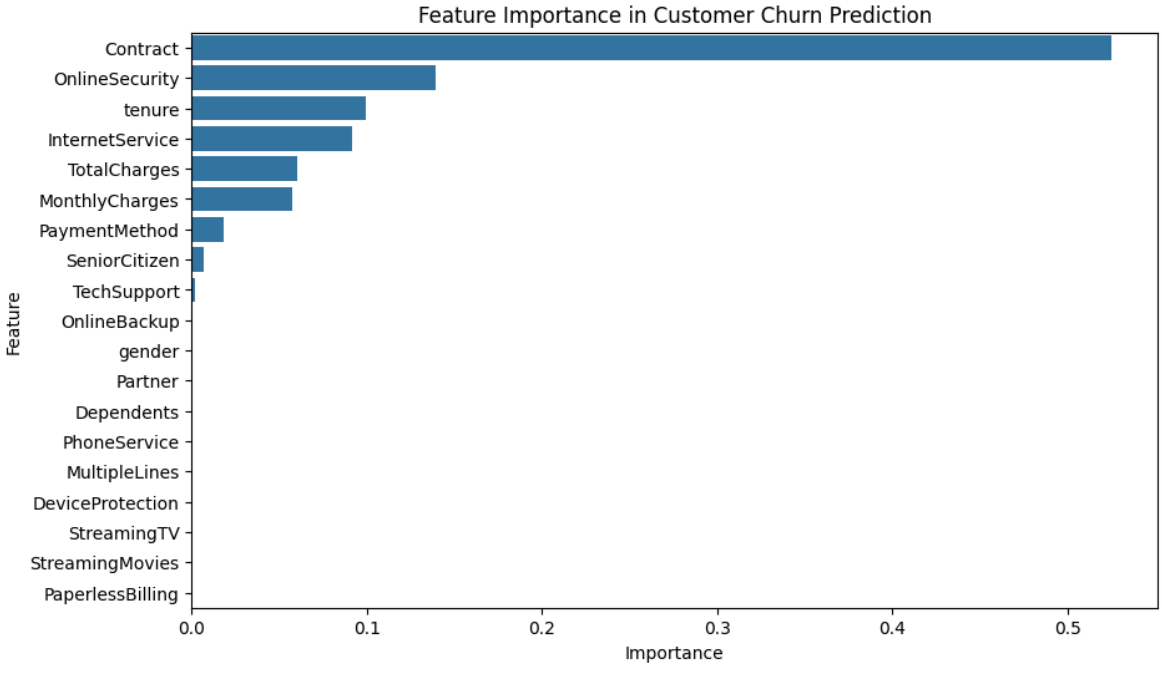

Confusion Matrix:
[[899 136]
 [167 207]]
```



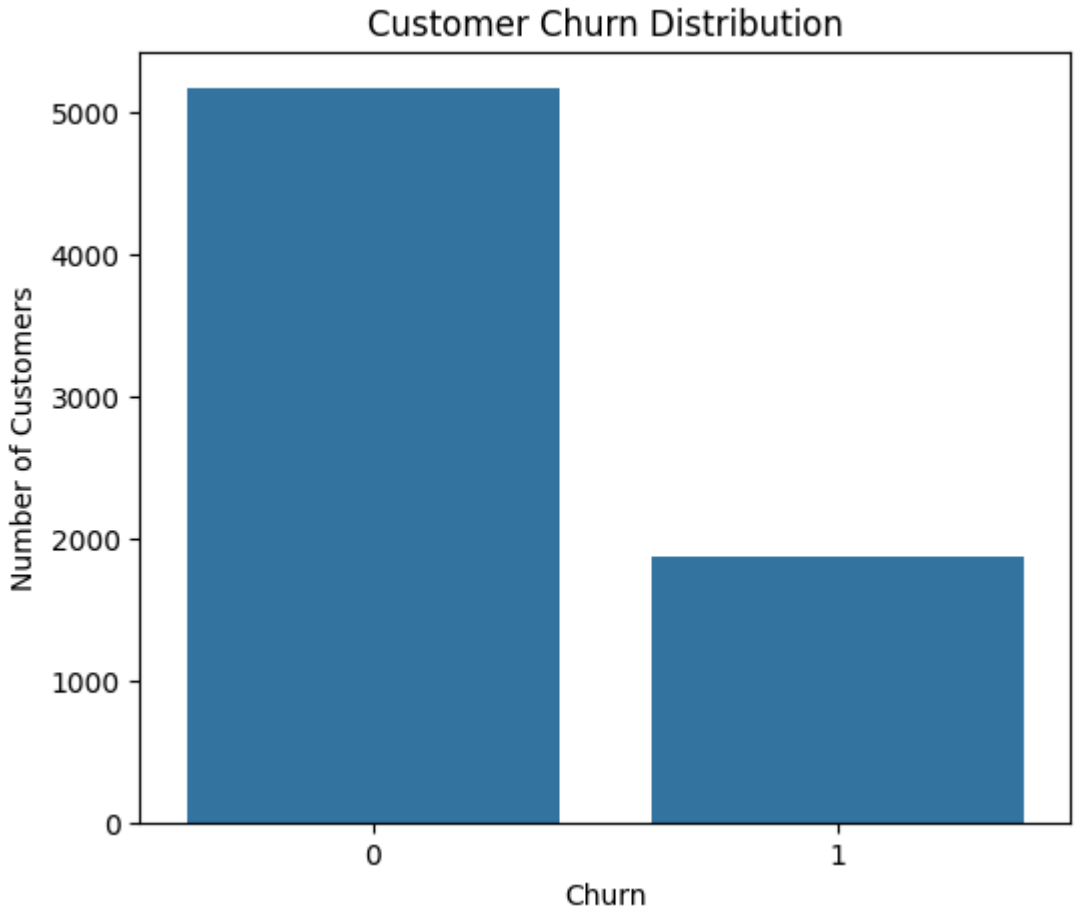


Feature Importance:

	Feature	Importance
14	Contract	0.525341
8	OnlineSecurity	0.138973
4	tenure	0.099174
7	InternetService	0.091187
18	TotalCharges	0.060525
17	MonthlyCharges	0.057434
16	PaymentMethod	0.018331
1	SeniorCitizen	0.006759
11	TechSupport	0.002142
9	OnlineBackup	0.000135
0	gender	0.000000
2	Partner	0.000000
3	Dependents	0.000000
5	PhoneService	0.000000
6	MultipleLines	0.000000
10	DeviceProtection	0.000000
12	StreamingTV	0.000000
13	StreamingMovies	0.000000
15	PaperlessBilling	0.000000



```
Customer Churn Distribution:
Churn
0    5174
1    1869
Name: count, dtype: int64
```



Customers Predicted to Churn:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	\
2280	0	1	0	0	8	1	
4460	1	0	1	0	18	1	
2136	1	0	0	1	15	1	
6529	1	1	0	0	2	1	
5389	0	0	1	0	51	1	
761	0	0	0	0	22	1	
5547	0	0	0	0	3	1	
1838	1	1	0	0	54	1	
6979	1	0	0	1	1	0	
346	0	0	0	0	2	1	
6633	0	0	0	0	1	1	
2519	1	0	1	0	2	1	
3510	0	0	0	0	8	1	
2246	0	0	1	1	1	1	
6304	0	0	0	0	9	1	
5300	0	0	1	1	1	1	
290	1	1	0	0	24	1	
642	1	0	0	0	1	1	
3856	1	0	1	0	61	1	
6314	0	0	0	0	6	1	

	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	...	\
2280	2	1	0	0	...	
4460	0	1	0	0	...	
2136	2	1	0	2	...	
6529	2	1	0	0	...	
5389	0	1	0	0	...	
761	0	1	0	0	...	
5547	0	1	0	0	...	
1838	2	1	0	2	...	
6979	1	0	0	0	...	
346	0	1	0	0	...	
6633	2	1	0	0	...	
2519	2	1	0	0	...	
3510	2	1	0	0	...	
2246	2	1	2	0	...	
6304	0	1	0	0	...	
5300	0	1	0	0	...	
290	2	1	0	0	...	
642	0	1	0	0	...	
3856	2	1	0	2	...	
6314	2	1	0	0	...	

	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	\
2280	2	2	2	0	1	
4460	2	0	0	0	0	
2136	0	2	0	0	1	
6529	0	0	0	0	1	
5389	0	0	0	0	0	
761	0	2	2	0	0	
5547	0	2	2	0	1	
1838	0	0	2	0	1	
6979	0	0	0	0	0	
346	0	2	2	0	1	
6633	0	0	0	0	1	
2519	0	2	0	0	1	
3510	0	0	0	0	1	
2246	0	2	2	0	1	

```
6304            0            2                0        0                1
5300            0            2                0        0                1
290             0            0                0        0                1
642             0            2                2        0                1
3856            2            2                2        0                1
6314            0            0                0        0                1

      PaymentMethod  MonthlyCharges  TotalCharges  Actual Churn  \
2280              1          100.15        908.55             0
4460              2           78.20       1468.75             0
2136              2           91.00       1430.05             0
6529              3           74.85        156.40             1
5389              3           69.15       3649.60             0
761               2           89.25       1907.85             1
5547              2           89.45        240.45             1
1838              2           90.05       4931.80             0
6979              3           24.20         24.20             0
346               2           90.40        168.20             1
6633              2           74.50         74.50             1
2519              2           91.45        171.45             0
3510              2           75.25        576.70             0
2246              2          102.45        102.45             1
6304              0           77.65        714.15             1
5300              3           80.05         80.05             1
290               2           76.10       1712.70             1
642               2           89.55         89.55             1
3856              2          114.10       7132.15             0
6314              3           73.85        401.30             0

      Predicted Churn
2280                1
4460                1
2136                1
6529                1
5389                1
761                 1
5547                1
1838                1
6979                1
346                 1
6633                1
2519                1
3510                1
2246                1
6304                1
5300                1
290                 1
642                 1
3856                1
6314                1

[20 rows x 21 columns]

Final Result:
Decision Tree Customer Churn Prediction completed successfully.
Model Accuracy: 78.50%
Precision: 60.35%
Recall: 55.35%
F1 Score: 57.74%
```

In [ ]:

```
#EXPERIMENT NO. 9

import pandas as pd
import matplotlib.pyplot as plt

# Load Dataset
df = pd.read_csv("superstore.csv", encoding="latin1")

print("Dataset Loaded Successfully")
print("Dataset Shape:", df.shape)

# Sales by Category
category_sales = df.groupby("Category")["Sales"].sum()

plt.figure(figsize=(6,4))
plt.bar(category_sales.index, category_sales.values)
plt.title("Sales by Category")
plt.xlabel("Category")
plt.ylabel("Sales")
plt.show()

# Sales by Region
region_sales = df.groupby("Region")["Sales"].sum()

plt.figure(figsize=(7,4))
plt.bar(region_sales.index, region_sales.values)
plt.title("Sales by Region")
plt.xlabel("Region")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.show()

# Sales by Segment
segment_sales = df.groupby("Segment")["Sales"].sum()

plt.figure(figsize=(6,4))
plt.bar(segment_sales.index, segment_sales.values)
plt.title("Sales by Segment")
plt.xlabel("Segment")
plt.ylabel("Sales")
plt.show()

# Top 10 Countries
country_sales = df.groupby("Country")["Sales"].sum().sort_values(ascending=False).h

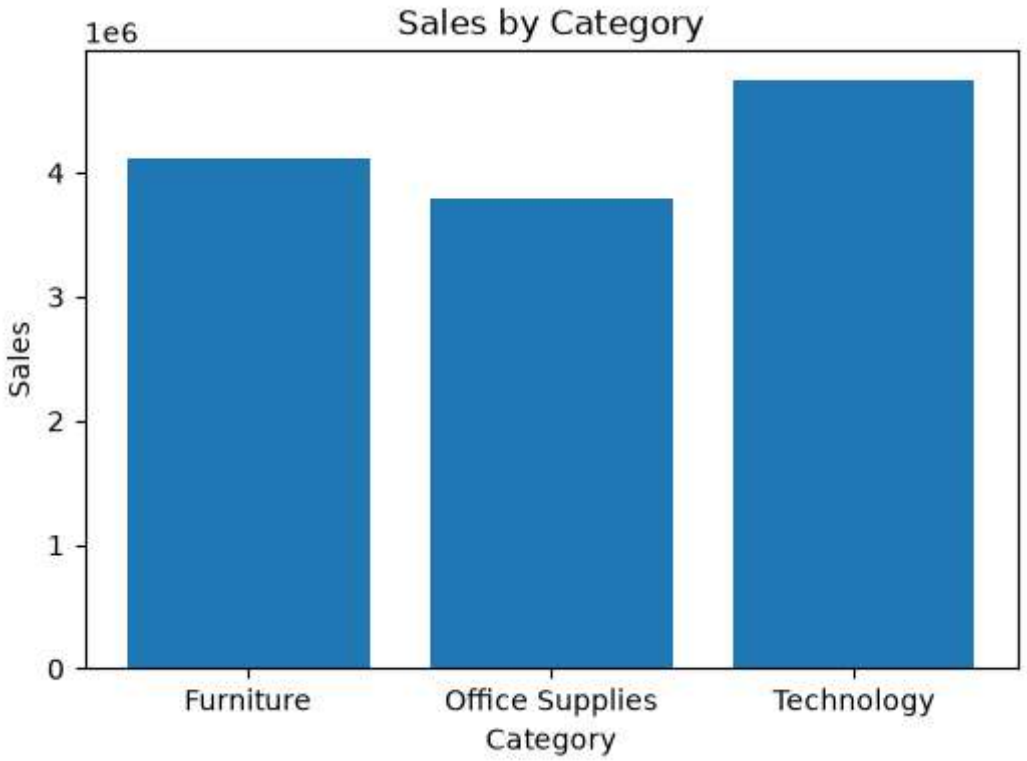
plt.figure(figsize=(8,5))
plt.barh(country_sales.index, country_sales.values)
plt.title("Top 10 Countries by Sales")
plt.xlabel("Sales")
plt.ylabel("Country")
plt.gca().invert_yaxis()
plt.show()

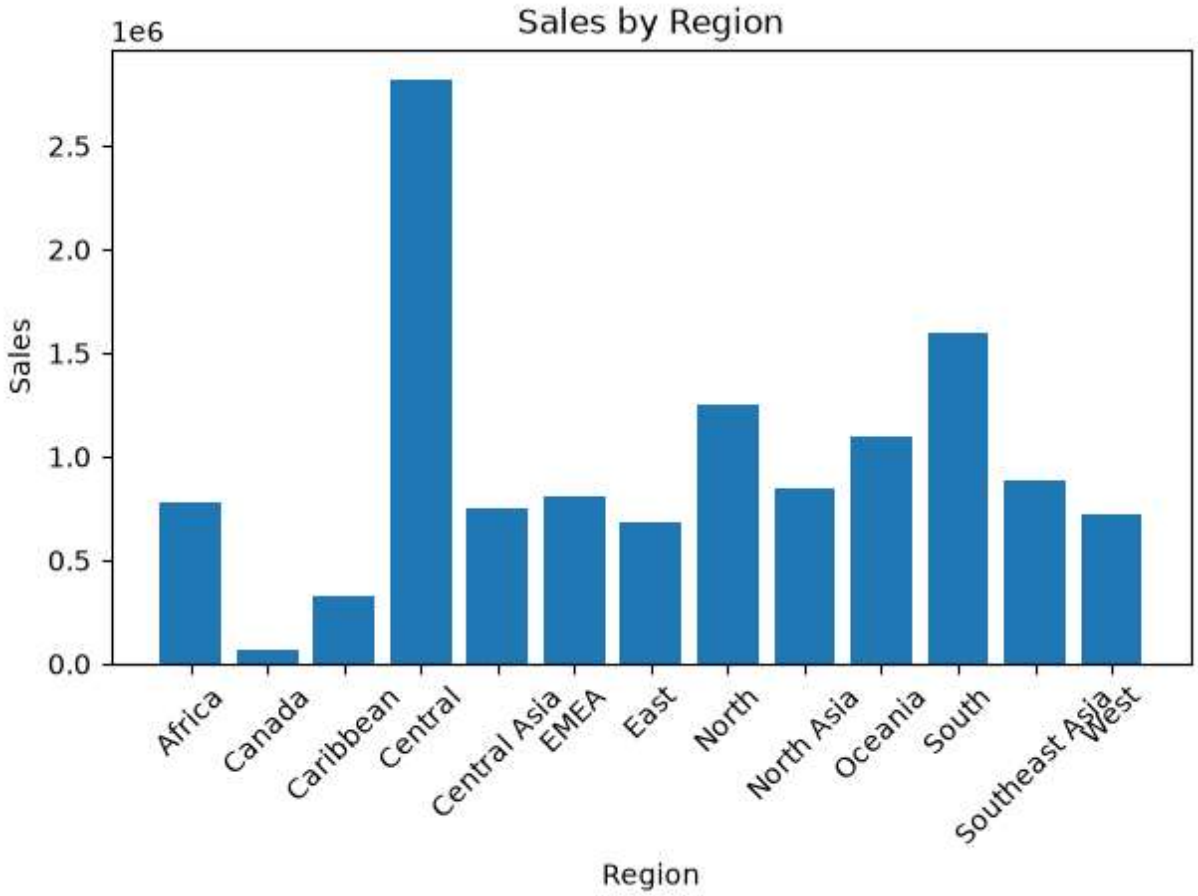
# Key Insights
```

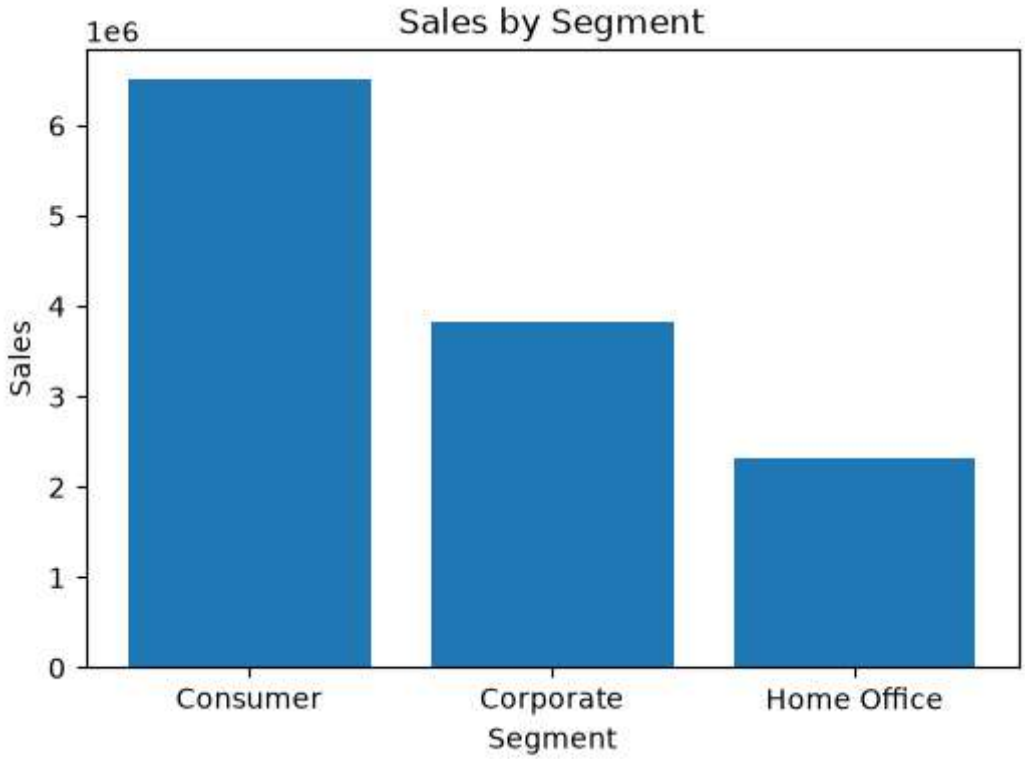
```
print("\n===== KEY INSIGHTS =====")
print("Total Sales:", round(df["Sales"].sum(), 2))
print("Total Profit:", round(df["Profit"].sum(), 2))
print("Top Category:", category_sales.idxmax())
print("Top Region:", region_sales.idxmax())
print("Top Segment:", segment_sales.idxmax())

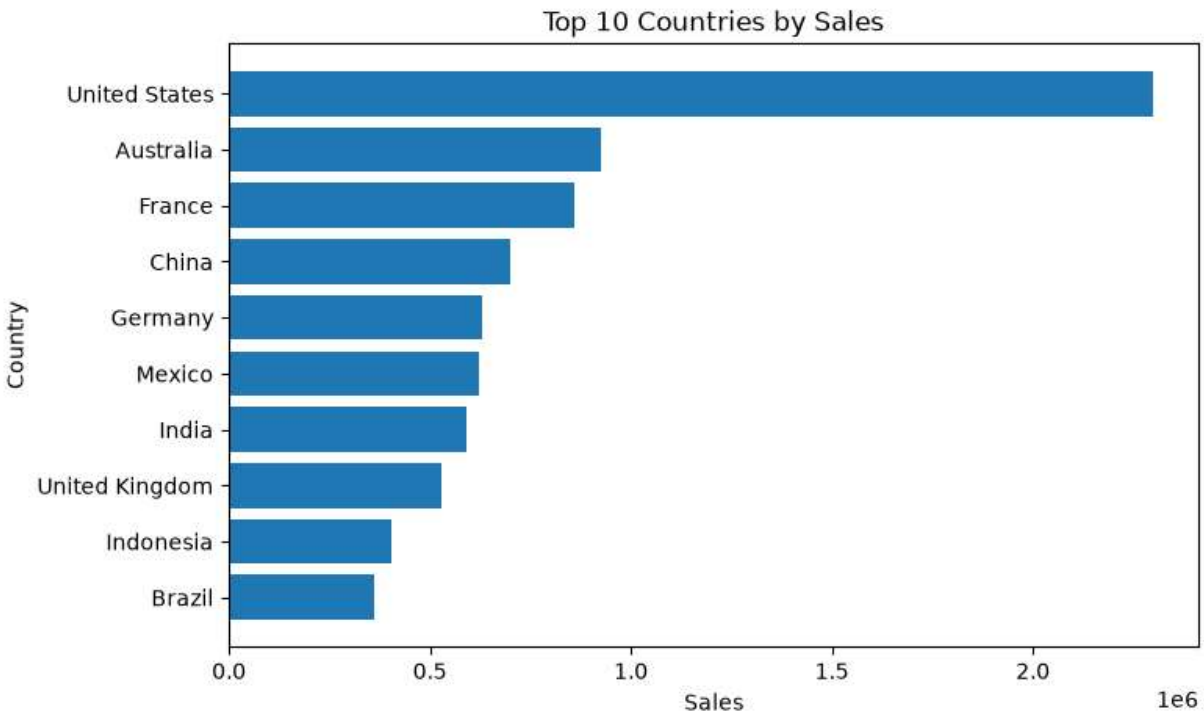
print("\nExperiment No. 9 Completed Successfully")
```

```
Dataset Loaded Successfully
Dataset Shape: (51290, 27)
```




Sales by Region
1e6
2.5
2.0
1.5
1.0
0.5
0.0
Sales
Africa
Canada
Caribbean
Central
Central Asia
EMEA
East
North
North Asia
Oceania
South
Southeast Asia
West
Region


Sales by Segment
1e6
6
5
4
3
2
1
0
Sales
Consumer
Corporate
Home Office
Segment




```
===== KEY INSIGHTS =====
Total Sales: 12642905
Total Profit: 1467457.29
Top Category: Technology
Top Region: Central
Top Segment: Consumer

Experiment No. 9 Completed Successfully
```

In [ ]:
```
# Experiment No. 9

## Experiment Name
Data Storytelling and Business Insight Generation using Interactive Visualizations

## Aim
To create a compelling data-driven story by analyzing a real-world dataset, develop

## Dataset Used
Netflix Movies and TV Shows Dataset

## Objective
To analyze Netflix content using data visualization and identify useful trends and
```

QUESTIONS AND ANSWERS

Q1. What is data storytelling? How is it different from data visualization?

Answer: Data storytelling combines data, visualizations, and explanations to communicate insights. Data visualization mainly represents data through charts and graphs.

Q2. Why is storytelling important in business analytics and decision-making?

Answer: Storytelling makes complex data easy to understand and helps stakeholders identify trends and make informed decisions.

Q3. What are the essential components of an effective data story?

Answer: The main components are data, visualizations, insights, context, and a clear narrative.

Q4. How do Key Performance Indicators (KPIs) enhance business reporting?

Answer: KPIs provide important measurable values that help track performance and understand business results.

Q5. Why should visualizations be arranged in a logical sequence?

Answer: A logical sequence makes the analysis easier to follow and clearly explains the problem, findings, and conclusions.

Q6. What factors should be considered while selecting visualizations?

Answer: The type of data, purpose of analysis, audience, readability, and comparison requirements should be considered.

Q7. Explain how dashboards and storytelling complement each other in Business Intelligence.

Answer: Dashboards provide interactive visual information, while storytelling explains the meaning of the information and highlights important insights.

Q8. What challenges may arise while communicating analytical insights to non-technical stakeholders?

Answer: Common challenges include technical terms, complex charts, too much information, and difficulty understanding statistical results.

Q9. Give two real-world examples where data storytelling has influenced business or policy decisions.

Answer: Netflix uses viewing data to understand audience preferences and plan content. Governments use data stories to understand public trends and support policy decisions.

Q10. How can effective data storytelling improve strategic planning and organizational performance?

Answer: It helps organizations understand trends, identify opportunities and risks, and make better data-driven decisions.

In [ ]:

```
# Experiment No. 10

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from scipy.stats import chi2_contingency

# 1. DATA COLLECTION AND PREPROCESSING

df = pd.read_csv("superstore.csv", encoding="latin1")

df = df.drop_duplicates()

print("========== DATASET INFORMATION ==========")
print("Dataset Shape :", df.shape)
print("Missing Values:", df.isnull().sum().sum())

# 2. EXPLORATORY DATA ANALYSIS

category_count = df["Category"].value_counts()

plt.figure(figsize=(7,5))
plt.bar(category_count.index, category_count.values)
plt.title("Products by Category")
plt.xlabel("Category")
plt.ylabel("Number of Products")
plt.show()

# 3. CORRELATION ANALYSIS

correlation = df[["Sales", "Quantity", "Discount", "Profit"]].corr()

plt.figure(figsize=(6,5))
sns.heatmap(correlation, annot=True, cmap="Blues", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()

# 4. STATISTICAL ANALYSIS

table = pd.crosstab(df["Category"], df["Region"])
chi2, p, dof, expected = chi2_contingency(table)

# 5. PREDICTIVE MODELING

data = df.dropna(subset=["Sales", "Quantity", "Discount", "Profit", "Category"])

X = data[["Sales", "Quantity", "Discount", "Profit"]]

y = data["Category"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

model = DecisionTreeClassifier(
    max_depth=5,
    random_state=42
)

model.fit(X_train, y_train)

pred = model.predict(X_test)

# 6. MODEL EVALUATION

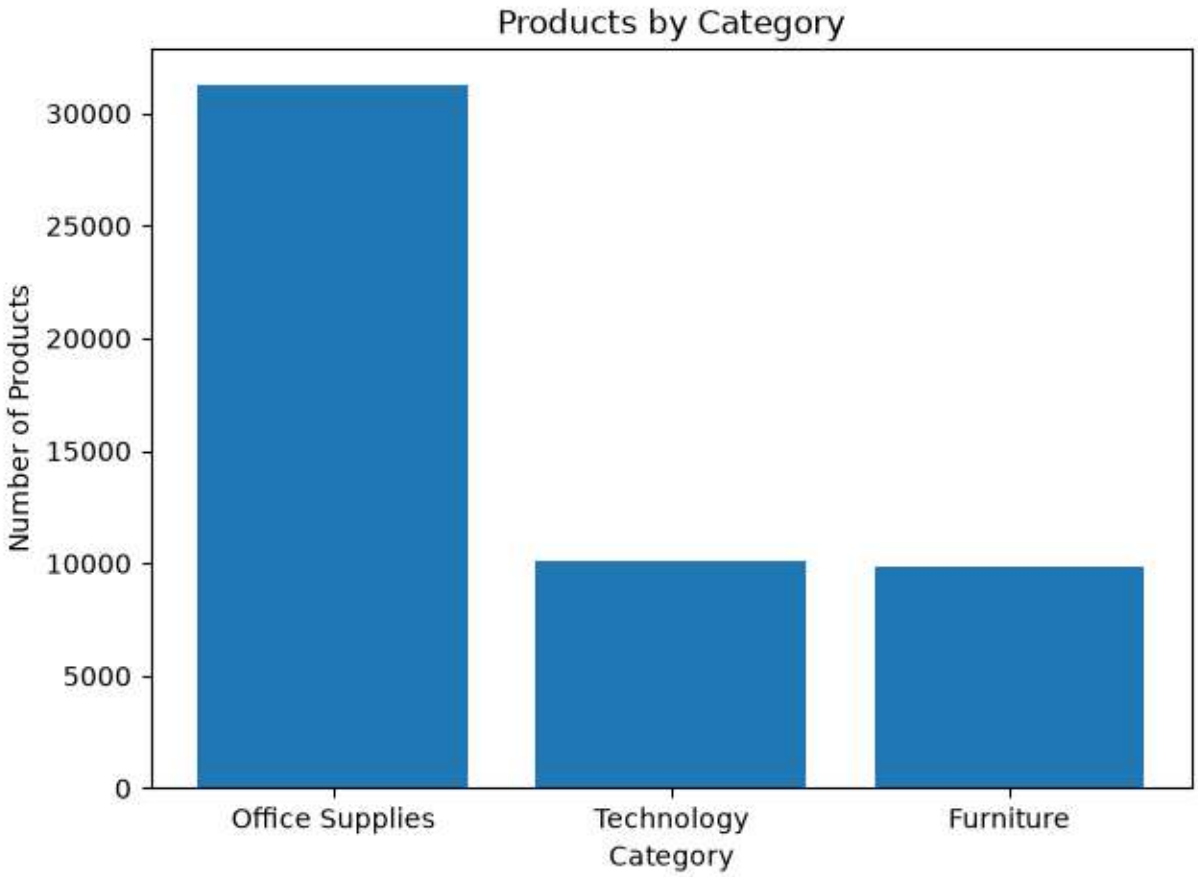
print("\n========== MODEL PERFORMANCE ==========")
print("Accuracy :", round(accuracy_score(y_test, pred), 3))
print("Precision:", round(precision_score(y_test, pred, average="weighted"), 3))
print("Recall   :", round(recall_score(y_test, pred, average="weighted"), 3))
print("F1-Score :", round(f1_score(y_test, pred, average="weighted"), 3))

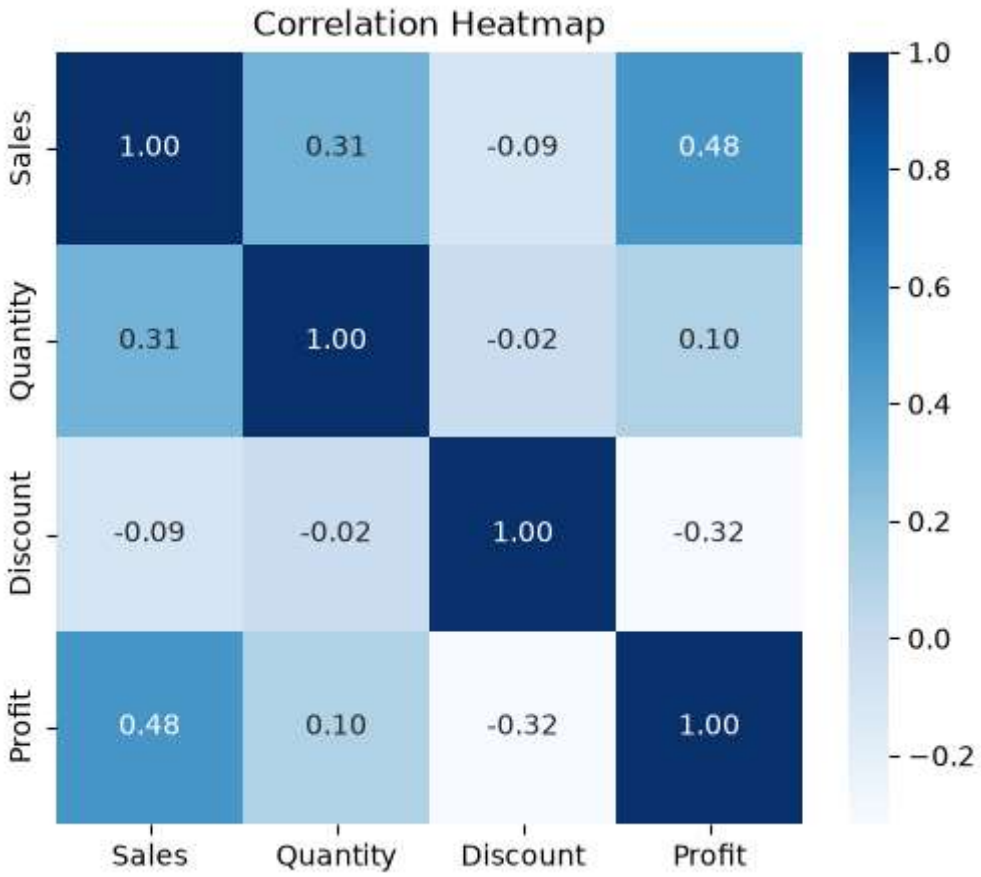
# 7. BUSINESS INSIGHTS

print("\n========== BUSINESS INSIGHTS ==========")
print("Total Sales :", round(df["Sales"].sum(), 2))
print("Total Profit:", round(df["Profit"].sum(), 2))
print("Top Category:", df.groupby("Category")["Sales"].sum().idxmax())
print("Top Region  :", df.groupby("Region")["Sales"].sum().idxmax())
print("Chi-Square p:", round(p, 4))

print("\nExperiment No. 10 Completed Successfully")
```

```
========== DATASET INFORMATION ==========
Dataset Shape : (51290, 27)
Missing Values: 0
```


Products by Category
Number of Products
30000
25000
20000
15000
10000
5000
0
Office Supplies
Technology
Furniture
Category


Correlation Heatmap
Sales
Quantity
Discount
Profit
1.00
0.31
-0.09
0.48
0.31
1.00
-0.02
0.10
-0.09
-0.02
1.00
-0.32
0.48
0.10
-0.32
1.00
Sales
Quantity
Discount
Profit
1.0
0.8
0.6
0.4
0.2
0.0
−0.2

```
========== MODEL PERFORMANCE ==========
Accuracy : 0.69
Precision: 0.665
Recall   : 0.69
F1-Score : 0.667

========== BUSINESS INSIGHTS ==========
Total Sales : 12642905
Total Profit: 1467457.29
Top Category: Technology
Top Region  : Central
Chi-Square p: 0.0

Experiment No. 10 Completed Successfully
```

EXPERIMENT NO. 10

Experiment Name: End-to-End Data Analytics Project: Business Intelligence and Predictive Analytics using a Real-World Dataset

Aim: To implement a complete data analytics workflow on a real-world dataset by performing data collection, preprocessing, exploratory data analysis, visualization, statistical analysis, predictive modeling, and presentation of actionable business insights.

QUESTIONS AND ANSWERS

Q1. What are the major stages involved in an end-to-end data analytics project?

Answer: The major stages are data collection, preprocessing, EDA, statistical analysis, visualization, machine learning, evaluation, and reporting.

Q2. Why is data preprocessing important?

Answer: Preprocessing cleans the data by handling missing values, duplicates, and incorrect data. It improves the quality of analysis and model performance.

Q3. How does EDA contribute to model development?

Answer: EDA helps identify patterns, relationships, trends, and unusual values. It helps in selecting suitable features and models.

Q4. What factors should be considered while selecting a machine learning algorithm?

Answer: The problem type, dataset size, data type, accuracy requirements, training time, and model interpretability should be considered.

Q5. Why is model evaluation essential?

Answer: Model evaluation measures how well a model performs on unseen data using suitable performance metrics.

Q6. How do Business Intelligence tools complement machine learning?

Answer: BI tools such as Tableau and Power BI present data and model results using interactive dashboards and visualizations.

Q7. What challenges are commonly encountered with real-world datasets?

Answer: Common challenges include missing values, duplicates, outliers, inconsistent data, and categorical variables.

Q8. How can analytical insights be converted into business recommendations?

Answer: Important trends, opportunities, and risks are identified from the analysis and converted into specific actions.

Q9. Why is data storytelling important?

Answer: Data storytelling explains complex results in a simple way and helps users understand findings for better decision-making.

Q10. Suggest future improvements for the analytics solution.

Answer: Future improvements can include feature engineering, hyperparameter tuning, advanced machine learning algorithms, larger datasets, and interactive dashboards.

Labsheet - 3

Statistical Analysis and Hypothesis Testing using Python

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import ttest_ind
from scipy.stats import f_oneway
import statsmodels.api as sm
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# Step 1: Load dataset
file_path = "WA_Fn-UseC_-HR-Employee-Attrition-selected-columns.csv"
df = pd.read_csv(file_path)
print("Dataset loaded successfully!")
print("\nDataset Shape:")
print(df.shape)
print("\nFirst 5 Records:")
print(df.head())

# Step 2: Basic information
print("\nDataset Information:")
df.info()
print("\nMissing Values:")
print(df.isnull().sum())
print("\nColumn Names:")
print(df.columns.tolist())

# Step 3: Descriptive Statistics
numerical_columns = [
    "Age",
    "DailyRate",
    "DistanceFromHome",
    "Education",
    "EmployeeCount",
    "EmployeeNumber"
]
print("\nMean:")
print(df[numerical_columns].mean())
print("\nMedian:")
print(df[numerical_columns].median())
print("\nMode:")
print(df[numerical_columns].mode().iloc[0])
print("\nVariance:")
print(df[numerical_columns].var())
print("\nStandard Deviation:")
print(df[numerical_columns].std())
print("\nComplete Descriptive Statistics:")
print(df[numerical_columns].describe())

# Step 4: Pearson Correlation
correlation = df[numerical_columns].corr()
print("\nPearson Correlation Matrix:")
print(correlation)
plt.figure(figsize=(10, 7))
```

```
sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)
plt.title("Pearson Correlation Matrix")
plt.tight_layout()
plt.show()

# Step 5: Hypothesis
print("\nHypothesis Testing")
print("H0: There is no significant difference in Age")
print("    between employees who leave and employees who stay.")
print("\nH1: There is a significant difference in Age")
print("    between employees who leave and employees who stay.")

# Step 6: Independent Sample T-Test
age_left = df[
    df["Attrition"] == "Yes"
]["Age"]
age_stayed = df[
    df["Attrition"] == "No"
]["Age"]
t_statistic, p_value = ttest_ind(
    age_left,
    age_stayed,
    equal_var=False
)
print("\nIndependent Sample T-Test")
print("Mean Age - Employees who left:")
print(age_left.mean())
print("Mean Age - Employees who stayed:")
print(age_stayed.mean())
print("T-Statistic:", t_statistic)
print("P-Value:", p_value)
alpha = 0.05
if p_value < alpha:
    print("\nResult: Reject H0.")
    print("There is a statistically significant difference")
    print("in Age between employees who left and stayed.")
else:
    print("\nResult: Fail to reject H0.")
    print("There is no statistically significant difference")
    print("in Age between employees who left and stayed.")

# Step 7: One-Way ANOVA
print("\nOne-Way ANOVA")
print("Testing Age differences across Departments.")
department_groups = []
departments = df["Department"].unique()
for department in departments:
    group = df[
        df["Department"] == department
    ]["Age"]
    department_groups.append(group)
f_statistic, anova_p_value = f_oneway(
    *department_groups
)
print("Departments:")
```

```
print(departments)
print("F-Statistic:", f_statistic)
print("P-Value:", anova_p_value)
if anova_p_value < alpha:
    print("\nResult: Reject H0.")
    print("There is a significant difference in Age")
    print("among the different departments.")
else:
    print("\nResult: Fail to reject H0.")
    print("There is no significant difference in Age")
    print("among the different departments.")

# Department vs Age visualization
plt.figure(figsize=(8, 6))
sns.boxplot(
    data=df,
    x="Department",
    y="Age"
)
plt.title("Age Distribution Across Departments")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()

# Step 8: Simple Linear Regression
# Independent variable = DistanceFromHome
# Dependent variable = DailyRate
X = df[["DistanceFromHome"]]
y = df["DailyRate"]
model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)
slope = model.coef_[0]
intercept = model.intercept_
r2 = r2_score(
    y,
    y_pred
)
print("\nLinear Regression")
print("Independent Variable: DistanceFromHome")
print("Dependent Variable: DailyRate")
print("Slope:", slope)
print("Intercept:", intercept)
print("R² Score:", r2)
# Regression Plot
plt.figure(figsize=(8, 6))
plt.scatter(
    X,
    y,
    alpha=0.5
)
plt.plot(
    X,
    y_pred
)
plt.xlabel("Distance From Home")
plt.ylabel("Daily Rate")
plt.title(
    "Linear Regression: Distance From Home vs Daily Rate"
)
```

```
plt.tight_layout()
plt.show()

# Step 9: Statsmodels Regression
X_stats = sm.add_constant(X)
regression_model = sm.OLS(
    y,
    X_stats
).fit()
print("\nStatsmodels Regression Summary:")
print(regression_model.summary())
print("\nRegression Coefficients:")
print(regression_model.params)
print("\nP-Values:")
print(regression_model.pvalues)
print("\n95% Confidence Intervals:")
print(regression_model.conf_int())
print("\nR² Score:")
print(regression_model.rsquared)

# Step 10: Final Summary
print("\nFinal Summary")
print("1. Descriptive statistics were calculated.")
print("2. Pearson correlation analysis was performed.")
print("3. Independent Sample T-Test was performed.")
print("4. One-Way ANOVA was performed.")
print("5. Linear Regression was performed.")
print("6. P-values were analyzed.")
print("7. Confidence intervals were calculated.")
print("8. R² score was calculated.")
print("\nStatistical analysis helps identify")
print("relationships, differences, and trends")
print("in real-world datasets.")
print("\nExperiment No. 3 completed successfully!")
```

```
Dataset loaded successfully!

Dataset Shape:
(1470, 10)

First 5 Records:
   Age Attrition     BusinessTravel  DailyRate              Department  \
0   41       Yes      Travel_Rarely       1102                   Sales
1   49        No  Travel_Frequently        279  Research & Development
2   37       Yes      Travel_Rarely       1373  Research & Development
3   33        No  Travel_Frequently       1392  Research & Development
4   27        No      Travel_Rarely        591  Research & Development

   DistanceFromHome  Education EducationField  EmployeeCount  EmployeeNumber
0                 1          2  Life Sciences              1               1
1                 8          1  Life Sciences              1               2
2                 2          2          Other              1               4
3                 3          4  Life Sciences              1               5
4                 2          1        Medical              1               7

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 10 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Age               1470 non-null   int64
 1   Attrition         1470 non-null   object
 2   BusinessTravel    1470 non-null   object
 3   DailyRate         1470 non-null   int64
 4   Department        1470 non-null   object
 5   DistanceFromHome  1470 non-null   int64
 6   Education         1470 non-null   int64
 7   EducationField    1470 non-null   object
 8   EmployeeCount     1470 non-null   int64
 9   EmployeeNumber    1470 non-null   int64
dtypes: int64(6), object(4)
memory usage: 115.0+ KB

Missing Values:
Age                 0
Attrition           0
BusinessTravel      0
DailyRate           0
Department          0
DistanceFromHome    0
Education           0
EducationField      0
EmployeeCount       0
EmployeeNumber      0
dtype: int64

Column Names:
['Age', 'Attrition', 'BusinessTravel', 'DailyRate', 'Department', 'DistanceFromHo
me', 'Education', 'EducationField', 'EmployeeCount', 'EmployeeNumber']

Mean:
Age                  36.923810
DailyRate           802.485714
DistanceFromHome      9.192517
```

```
Education              2.912925
EmployeeCount          1.000000
EmployeeNumber      1024.865306
dtype: float64

Median:
Age                    36.0
DailyRate             802.0
DistanceFromHome        7.0
Education               3.0
EmployeeCount           1.0
EmployeeNumber       1020.5
dtype: float64

Mode:
Age                   35.0
DailyRate            691.0
DistanceFromHome       2.0
Education              3.0
EmployeeCount          1.0
EmployeeNumber         1.0
Name: 0, dtype: float64

Variance:
Age                     83.455049
DailyRate           162819.593737
DistanceFromHome        65.721251
Education                1.048914
EmployeeCount            0.000000
EmployeeNumber      362433.299749
dtype: float64

Standard Deviation:
Age                    9.135373
DailyRate            403.509100
DistanceFromHome       8.106864
Education              1.024165
EmployeeCount          0.000000
EmployeeNumber       602.024335
dtype: float64

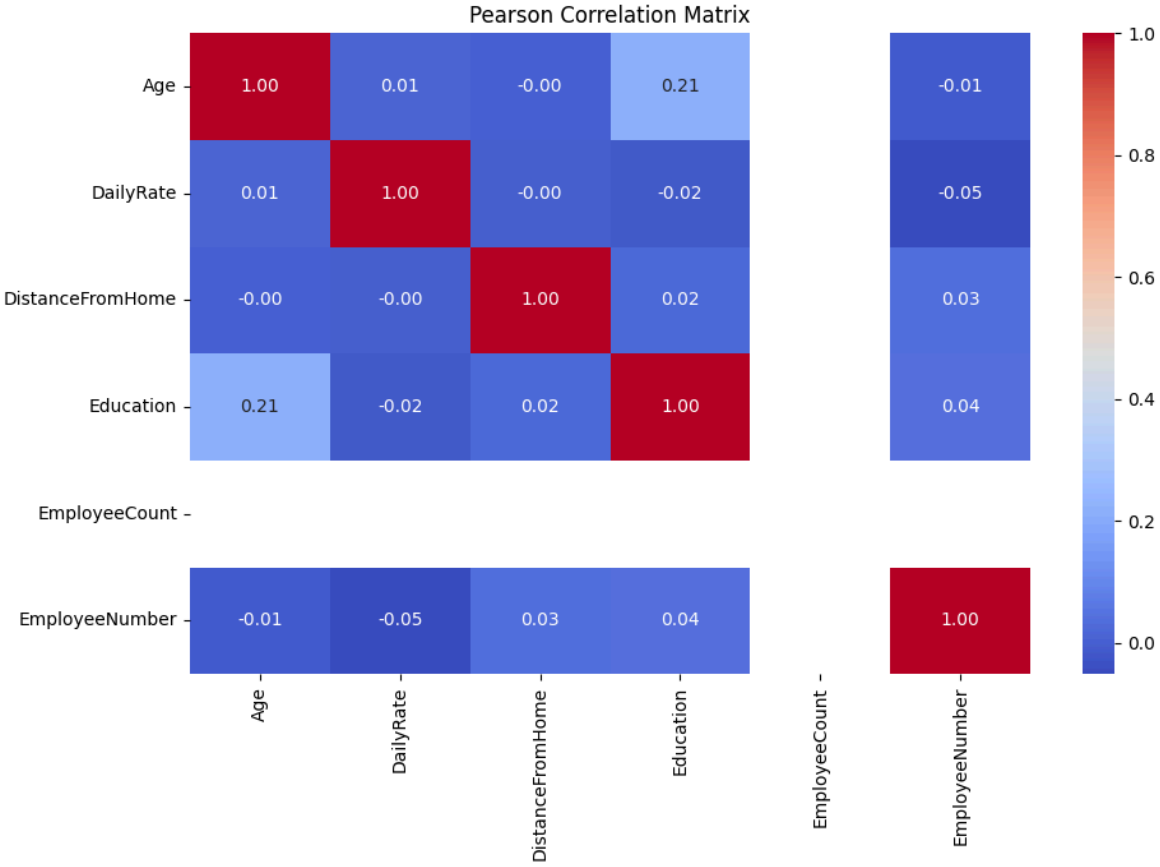
Complete Descriptive Statistics:
               Age    DailyRate  DistanceFromHome    Education  EmployeeCount  \
count  1470.000000  1470.000000       1470.000000  1470.000000         1470.0
mean     36.923810   802.485714          9.192517     2.912925            1.0
std       9.135373   403.509100          8.106864     1.024165            0.0
min      18.000000   102.000000          1.000000     1.000000            1.0
25%      30.000000   465.000000          2.000000     2.000000            1.0
50%      36.000000   802.000000          7.000000     3.000000            1.0
75%      43.000000  1157.000000         14.000000     4.000000            1.0
max      60.000000  1499.000000         29.000000     5.000000            1.0

       EmployeeNumber
count     1470.000000
mean      1024.865306
std        602.024335
min          1.000000
25%        491.250000
50%       1020.500000
75%       1555.750000
```

```
max       2068.000000

Pearson Correlation Matrix:
                        Age  DailyRate  DistanceFromHome  Education  \
Age                1.000000   0.010661         -0.001686   0.208034
DailyRate          0.010661   1.000000         -0.004985  -0.016806
DistanceFromHome  -0.001686  -0.004985          1.000000   0.021042
Education          0.208034  -0.016806          0.021042   1.000000
EmployeeCount           NaN        NaN               NaN        NaN
EmployeeNumber    -0.010145  -0.050990          0.032916   0.042070

                  EmployeeCount  EmployeeNumber
Age                         NaN       -0.010145
DailyRate                   NaN       -0.050990
DistanceFromHome            NaN        0.032916
Education                   NaN        0.042070
EmployeeCount               NaN             NaN
EmployeeNumber              NaN        1.000000
```



```
Hypothesis Testing
H0: There is no significant difference in Age
    between employees who leave and employees who stay.

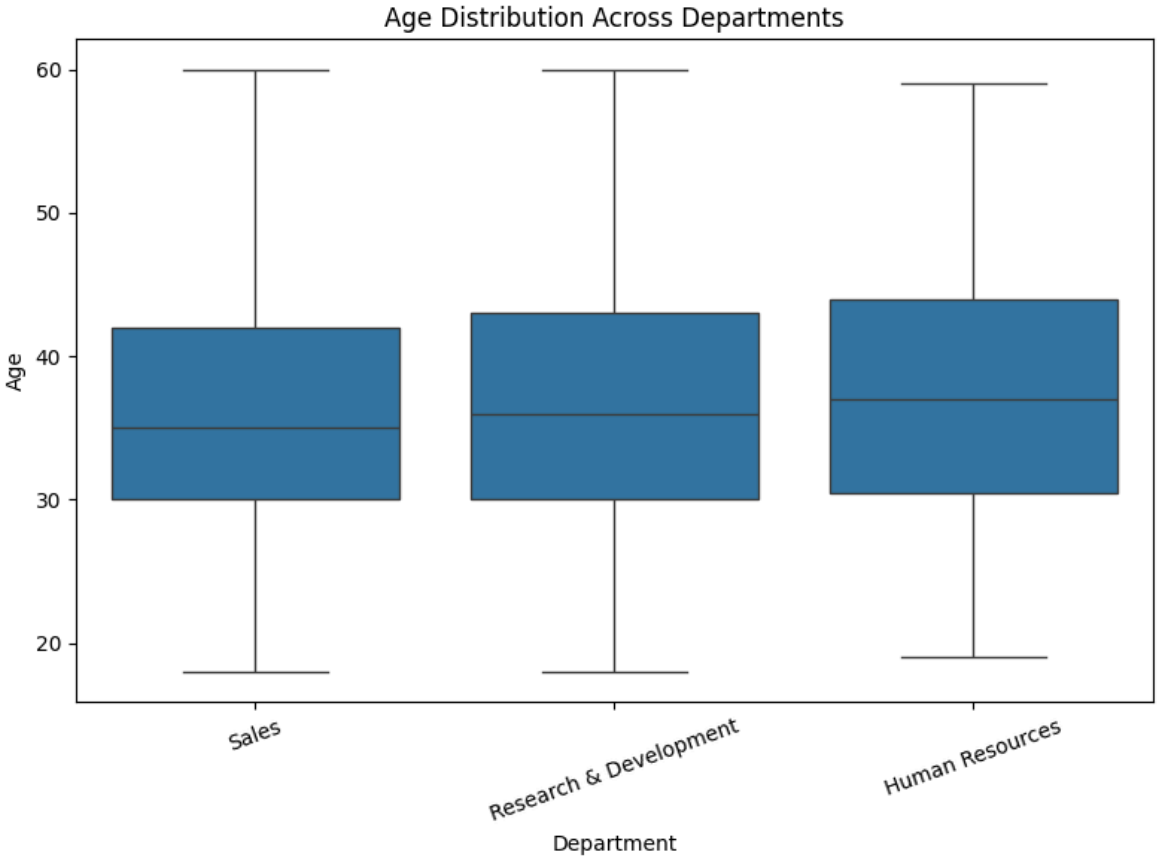
H1: There is a significant difference in Age
    between employees who leave and employees who stay.

Independent Sample T-Test
Mean Age - Employees who left:
33.607594936708864
Mean Age - Employees who stayed:
37.561232765612324
T-Statistic: -5.82801185398895
P-Value: 1.3797600649439775e-08

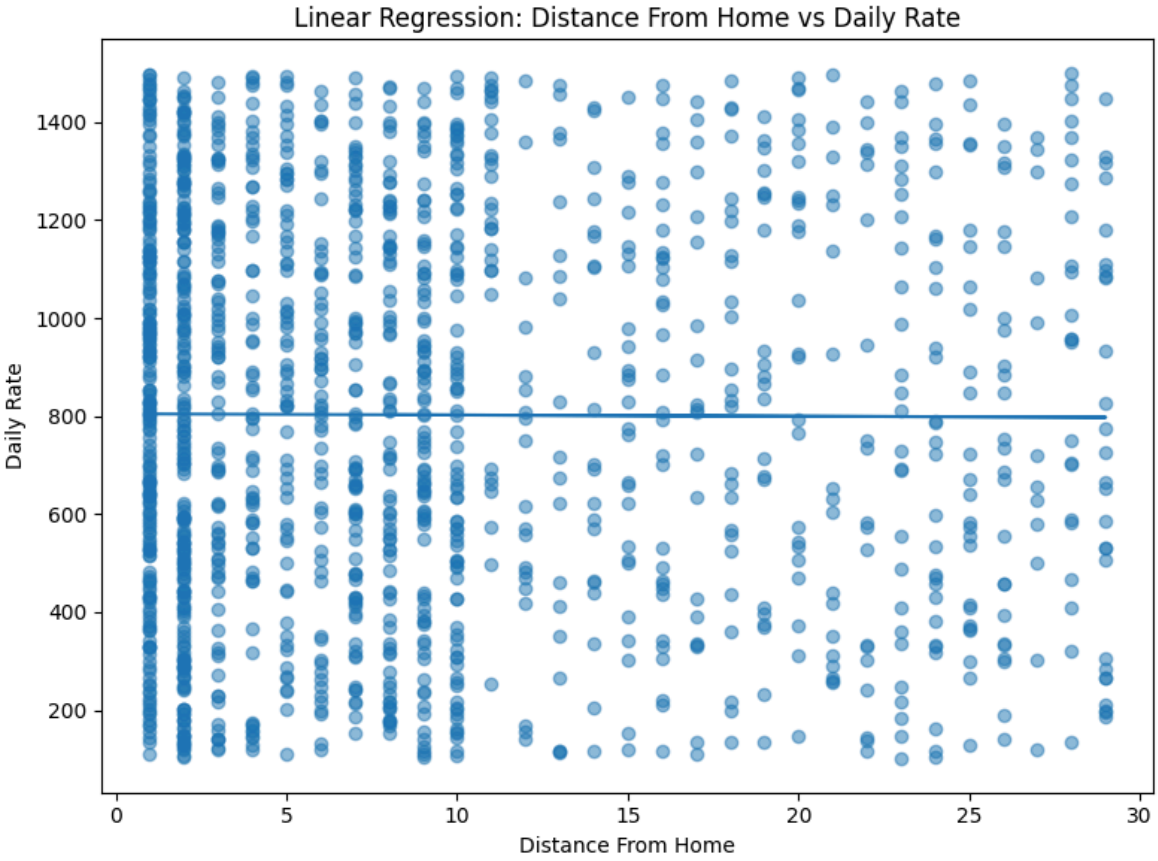
Result: Reject H0.
There is a statistically significant difference
in Age between employees who left and stayed.

One-Way ANOVA
Testing Age differences across Departments.
Departments:
['Sales' 'Research & Development' 'Human Resources']
F-Statistic: 0.7655031924976903
P-Value: 0.46528552999349515

Result: Fail to reject H0.
There is no significant difference in Age
among the different departments.
```



```
Linear Regression
Independent Variable: DistanceFromHome
Dependent Variable: DailyRate
Slope: -0.2481389696358105
Intercept: 804.766735984142
R² Score: 2.485358851878061e-05
```



```
Statsmodels Regression Summary:
                            OLS Regression Results
==============================================================================
Dep. Variable:              DailyRate   R-squared:                       0.000
Model:                            OLS   Adj. R-squared:                 -0.001
Method:                 Least Squares   F-statistic:                   0.03649
Date:                Mon, 07 Sep 2026   Prob (F-statistic):              0.849
Time:                        20:27:34   Log-Likelihood:                -10906.
No. Observations:                1470   AIC:                         2.182e+04
Df Residuals:                    1468   BIC:                         2.183e+04
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          804.7667     15.920     50.551      0.000     773.539     835.995
DistanceFromHome    -0.2481      1.299     -0.191      0.849      -2.796       2.300
==============================================================================
Omnibus:                     1173.556   Durbin-Watson:                   2.082
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               88.680
Skew:                          -0.003   Prob(JB):                     5.54e-20
Kurtosis:                       1.797   Cond. No.                         18.6
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Regression Coefficients:
const               804.766736
DistanceFromHome     -0.248139
dtype: float64

P-Values:
const               1.976263e-323
DistanceFromHome     8.485418e-01
dtype: float64

95% Confidence Intervals:
                            0           1
const             773.538836  835.994636
DistanceFromHome   -2.796367    2.300089

R² Score:
2.485358851878061e-05

Final Summary
1. Descriptive statistics were calculated.
2. Pearson correlation analysis was performed.
3. Independent Sample T-Test was performed.
4. One-Way ANOVA was performed.
5. Linear Regression was performed.
6. P-values were analyzed.
7. Confidence intervals were calculated.
8. R² score was calculated.
```

```
Statistical analysis helps identify
relationships, differences, and trends
in real-world datasets.

Experiment No. 3 completed successfully!
```

Question

1. What is the difference between descriptive statistics and inferential statistics?

Descriptive statistics summarize and describe data.

Inferential statistics use sample data to draw conclusions about a population.

Descriptive	Inferential
Summarizes data.	Draws conclusions.
Mean, median, mode.	t-test, ANOVA.
Describes observed data.	Tests hypotheses.

---

2. Explain the concepts of the Null Hypothesis (H_0) and Alternative Hypothesis (H_1).

Null Hypothesis (H_0): Assumes there is no significant difference or relationship.

Alternative Hypothesis (H_1): Assumes there is a significant difference or relationship.

Example:

H_0: There is no significant difference between two groups.

H_1: There is a significant difference between two groups.

---

3. What is a p-value? How is it used to make statistical decisions?

A **p-value** indicates how compatible the observed result is with the Null Hypothesis.

- **p-value < 0.05** → Reject H_0.
- **p-value ≥ 0.05** → Fail to reject H_0.

A small p-value provides stronger evidence against H_0.

---

4. Differentiate between a t-test and ANOVA. In which situations is each test applied?

t-test	ANOVA
Compares two groups.	Compares three or more groups.
Uses t-statistic.	Uses F-statistic.

t-test	ANOVA
Example: two employee groups.	Example: multiple departments.

---

5. What does the Pearson correlation coefficient indicate? What are its possible values?

The **Pearson correlation coefficient (r)** measures the strength and direction of a linear relationship between two numerical variables.

Its value ranges from **−1 to +1**.

- **+1** → Perfect positive correlation.
- **0** → No linear correlation.
- **−1** → Perfect negative correlation.

---

6. Explain the significance of R^2 (Coefficient of Determination) in Linear Regression.

R^2 shows the proportion of variation in the dependent variable explained by the regression model.

For example:

R^2 = 0.80 means the model explains about **80% of the variation** in the dependent variable for the fitted data.

A value closer to **1** generally indicates greater explanatory power.

---

7. Why is statistical analysis important before applying machine learning algorithms?

Statistical analysis helps to:

- Understand the dataset.
- Find relationships between variables.
- Detect unusual values.
- Identify important features.
- Check data assumptions.
- Improve data preprocessing and model development.

---

8. What assumptions should be satisfied before performing a t-test or ANOVA?

Important assumptions include:

- Observations should be independent.
- Data should be approximately normally distributed within groups.
- Variances should be reasonably similar for standard tests.
- The dependent variable should be numerical.

---

9. How can regression analysis help organizations in forecasting and decision-making?

Regression analysis helps organizations to:

- Identify relationships between variables.
- Predict future values.
- Forecast trends.
- Support planning.
- Make data-driven decisions.

For example, regression can be used to predict **sales, prices, or demand**.

---

10. Give two real-world applications where hypothesis testing is commonly used in data analytics.

1. Business and Marketing

Used to determine whether a new advertising campaign significantly affects sales or customer engagement.

2. Healthcare

Used to determine whether a treatment produces a significant difference compared with a comparison group.

Other applications include:

- Quality control.
- Finance.
- Education.
- A/B testing.